

Hoofdstuk 1: Javascript + Vite + Bootstrap projecttemplate.....	9
Hoofdstuk 2: Functions.....	12
Doel van dit hoofdstuk.....	12
Concept & uitleg.....	12
Voorbeeld in jouw template.....	13
Mini-opdracht.....	14
Begeleide oplossing.....	15
Hoofdstuk 3: Function Scoping.....	16
Concept & uitleg.....	16
Denkbeeld.....	17
Oefening: Function Scope Demo.....	17
Mini-opdracht.....	18
Begeleide oplossing.....	19
Wat je nu beheerst.....	19
Hoofdstuk 4: Named Functions.....	20
Theorie: Named Functions.....	20
Demo: Kleur tonen met named functions.....	20
Wat zagen we?.....	22
Mini-opdracht.....	22
Begeleide oplossing mini-opdracht.....	23
Samenvatting.....	24
Hoofdstuk 5: Functions & Strings in ES6 (Template Literals).....	24
Theorie: Template Literals.....	24
Real-life analogie.....	25
Demo: Welkomstbericht generator.....	25
Mini-opdracht.....	27
Mini-opdracht begeleide oplossing.....	27
Samenvatting.....	28
Hoofdstuk 6: ES6 Arrow Functions (=>).....	29
Theorie: Arrow Functions.....	29
Vergelijking.....	29
Let op.....	30
Oefening: Bereken + Toon (arrow style).....	30
Wat zag je?.....	31
Mini-opdracht.....	31
Oplossing mini-opdracht.....	32
Samenvatting.....	32

Hoofdstuk 7: ES6 Arrays	33
Theorie: Arrays.....	33
Vergelijking in real life.....	33
Demo: gastenlijst (array + UI).....	34
Mini-opdracht.....	35
Mini-opdracht oplossing.....	36
Samenvatting.....	37
Hoofdstuk 8: DOM Nodes & Root Nodes	37
Theorie	37
Denkbeeld	38
Demo: tekst zetten via root nodes.....	38
Wat gebeurde hier?.....	40
Mini-opdracht.....	40
Mini-opdracht oplossing.....	40
Samenvatting.....	41
Volgend hoofdstuk.....	41
Hoofdstuk 9: JavaScript Output Methodes.....	42
Theorie: Output Methodes.....	42
Demo oefening: 3 outputmethoden.....	43
Mini-opdracht.....	44
Mini-opdracht oplossing.....	45
Samenvatting.....	45
Hoofdstuk 10: document.write() en document.writeln().....	46
Theorie	46
Demo: Waarom je het niet gebruikt.....	47
Mini-opdracht.....	48
Mini-opdracht oplossing.....	48
Samenvatting.....	49
Hoofdstuk 11: innerHTML vs textContent vs innerText.....	49
Theorie	49
Veiligheid tip.....	50
Demo: drie output methoden	50
Mini-opdracht.....	52
Oplossing mini-opdracht.....	52
Samenvatting.....	53
Hoofdstuk 12: DOM Selectors.....	53
Theorie: DOM Selectors.....	53

Denkbeeld	54
Demo: lijst filteren met querySelectorAll.....	54
Mini-opdracht.....	55
Oplossing mini-opdracht.....	55
Samenvatting.....	56
Hoofdstuk 13: Andere DOM-element properties & acties.....	56
Theorie	56
Metafoor.....	57
Demo: focus highlight + child counter	57
Mini-opdracht.....	59
Mini-opdracht oplossing.....	59
Samenvatting.....	60
Hoofdstuk 14: Klassen manipuleren (classList)	61
Theorie	61
Demo: Highlight toggle + knop states.....	61
Mini-opdracht.....	63
Mini-opdracht oplossing.....	63
Samenvatting.....	64
Hoofdstuk 15: Events & Formulieren.....	64
Theorie: Wat is een Event?	64
Metafoor.....	65
Demo: Live naam weergave + form submit.....	65
Mini-opdracht.....	67
Mini-opdracht oplossing.....	67
Samenvatting.....	68
Hoofdstuk 16: Event Types & preventDefault() (Dieper) + Mouse Events.....	69
Theorie	69
Demo: Link + Box hover effect + Submit block.....	70
Mini-opdracht.....	71
Oplossing mini-opdracht.....	72
Samenvatting.....	72
Hoofdstuk 17: Mini-Project – Todo App (Events + Arrays + DOM).....	73
Uitleg.....	73
UI Design.....	73
index.html.....	74
main.js.....	74
Demo features die studenten leren.....	75

Mini-Opdracht.....	76
Samenvatting.....	76
Hoofdstuk 18: Objects & Date() in JavaScript.....	77
Theorie: Wat is een object?	77
Theorie: Date() object.....	77
Use case: Leeftijd berekenen.....	78
Demo: Leeftijd-calculator.....	78
Mini-opdracht.....	79
Mini-opdracht oplossing.....	80
Samenvatting.....	81
Hoofdstuk 19: Object Constructors & Inheritance (Intro).....	81
Theorie.....	82
Demo: User maken en tonen.....	82
Mini-opdracht.....	84
Oplossing mini-opdracht.....	84
Inheritance Light.....	85
Samenvatting.....	85
Hoofdstuk 20: ES6 Classes + extends + super.....	86
Theorie: Wat is een Class?.....	86
Inheritance met extends.....	87
Demo: User + Admin UI.....	87
Mini-opdracht.....	89
Samenvatting.....	89
Hoofdstuk 21: Mini Project – Product Catalog (Webshop Logica).....	90
Concept.....	90
Class Model.....	90
HTML UI.....	90
Styles (optioneel animatie).....	91
JavaScript.....	91
Test checklist.....	92
Mini-opdracht (uitbreiding).....	93
Samenvatting.....	93
Hoofdstuk 22: Browser Object Model (BOM)	94
Theorie.....	94
Veiligheid: popups & redirects.....	95
Demo: BOM Info Display.....	95
Mini-opdracht.....	96

Mini-opdracht oplossing.....	96
Samenvatting.....	97
Hoofdstuk 23: Cookies & LocalStorage.....	98
Demo: Naam onthouden.....	99
Demo 2: Array opslaan (favorietenlijst).....	100
Mini-opdracht.....	101
Samenvatting.....	102
Hoofdstuk 24: AJAX & API's Intro.....	103
Theorie.....	103
Voorbeeld van async denken.....	104
Demo: Data ophalen met fetch().....	104
Wat leerde je eigenlijk?.....	105
Mini-opdracht.....	105
Samenvatting.....	106
Hoofdstuk 25: JSON File Load + Local API Simulator.....	106
Wat is JSON?.....	106
JSON in Vite project plaatsen.....	107
UI: Lokaal JSON bestand laden.....	107
UI resultaat.....	108
Mini-opdracht.....	109
Mini-opdracht oplossing (docentversie).....	109
Samenvatting.....	110
Hoofdstuk 26: POST met Fetch (Fake API) & CRUD Intro.....	111
Theorie.....	111
Denk in stappen.....	111
Mini Fake API: Create Product.....	111
index.html.....	112
main.js.....	112
Wat werkt hier?.....	113
Mini-Opdracht.....	113
Mini-Opdracht Oplossing (docentversie).....	114
Samenvatting.....	114
Hoofdstuk 27: PUT & DELETE (Fake CRUD).....	115
Theorie.....	115
Mini Fake Admin Panel.....	115
index.html (blok toevoegen).....	115
main.js.....	116

Mini-opdracht.....	118
Samenvatting.....	118
Hoofdstuk 28: Echte HTTP Requests (fetch POST/PUT/DELETE) + API Testing Tools.....	119
Theorie.....	119
API Tools uitleg.....	119
Belangrijk.....	120
Demo 1: GET request (opfrisser).....	120
Demo 2: POST request (echt).....	120
Demo 3: DELETE request (echt).....	121
Wat studenten nu snappen.....	122
Mini-opdracht.....	122
Samenvatting.....	122
Hoofdstuk 29: Async/Await Deep Dive + UX voor Loading & Errors.....	123
Theory: Async/Await.....	123
UX Rules (professioneel).....	123
Demo: Fake API + Loading UI.....	124
Bonus: Retry button.....	125
Advanced Concepts (kort).....	125
Mini-opdracht.....	126
Mini-opdracht Oplossing (docentversie, samenvatting).....	126
Samenvatting.....	126
Hoofdstuk 30: Fetch Timeout, AbortController & Race Conditions.....	127
Theorie.....	127
UX Patterns.....	127
Demo: Cancel & Timeout.....	128
Wat werkt hier?.....	129
Race Condition Uitleg.....	129
Mini-opdracht.....	130
Docent-oplossing (verkort).....	130
Samenvatting.....	130
Hoofdstuk 31: Multiple Requests + Promise.all + Loading Skeletons.....	131
Theorie.....	131
Skeleton UI.....	131
Skeleton CSS.....	131
Demo: Parallel API Load (users + posts).....	132
UX Resultaat.....	133
Mini-opdracht.....	134

Samenvatting.....	134
Hoofdstuk 32: Modules, Imports & Professionele Projectstructuur.....	135
Waarom modules?.....	135
Nieuwe Project structuur.....	136
We bouwen: User Card Component + API Service.....	136
js/components/userCard.js.....	136
js/utils/format.js.....	136
js/main.js.....	137
index.html voor test.....	137
Wat je nu hebt gebouwd.....	137
Extra inzichten.....	137
Mini-opdracht.....	138
Docent-oplossing hints.....	138
Samenvatting.....	139
Hoofdstuk 33: Eindproject — Mini Shopping Dashboard.....	139
Project Concept.....	139
Structuur.....	140
data/products.json.....	140
Utilities.....	140
Components.....	141
Services.....	141
HTML sectie.....	142
js/main.js.....	142
Wat dit project demonstreert.....	143
Mini-opdracht (student).....	144
Wat studenten nu kunnen.....	144
Hoofdstuk 34: Developer Tools & Debugging Mastery.....	145
Theorie (kort & scherp).....	145
Demo 1 — Console power: table, group, fouten lezen.....	145
Demo 2 — Breakpoints, Watch & Call Stack.....	146
Demo 3 — Network: Throttling & Offline, Errors lezen.....	147
Mini-opdracht.....	149
Begeleide oplossing (met inline comments).....	149
Samenvatting.....	150
Hoofdstuk 35: ES6+ Advanced Patterns & Array Mastery.....	151
Theorie.....	151
Demo 1 — Destructuring + Spread + Rest.....	152

Demo 2 — map, filter, reduce.....	153
Immutability (zoals React).....	154
Mini-opdracht.....	154
Samenvatting.....	155
Hoofdstuk 36: Modern Forms + Validatie + UX Patterns.....	155
Theorie.....	155
Demo Form: Login + Live Feedback.....	156
Mini-Opdracht.....	157
Oplossing (docentversie).....	158
Samenvatting.....	158

Hoofdstuk 1: Javascript + Vite + Bootstrap projecttemplate

Doel van dit hoofdstuk

In dit hoofdstuk maken we een basisproject dat we **bij elke oefening** opnieuw gebruiken. Je leert:

- een Vite projectstructuur begrijpen
- Bootstrap via npm + SCSS integreren
- een testcomponent bouwen om te controleren of alles werkt
- een vaste oefen-template inzetten voor alle volgende hoofdstukken

Na dit hoofdstuk heb je een solide startpunt voor alle volgende JavaScript-oefeningen.

Concept en uitleg

Waarom Vite?

Vite is een moderne bundler met:

- bliksemsnelle development server
- native ES modules (super logisch voor modern JS)
- automatische refresh bij codewijzigingen
- simpele build-flow voor productie

Waarom Bootstrap via npm?

We gebruiken npm zodat we:

- Bootstrap als onderdeel van onze build hebben
- SCSS kunnen overschrijven en uitbreiden
- géén CDN nodig hebben
- professioneel en future-proof werken

Hoe gaan we oefenen?

Elke oefening start van dezelfde template:

- `index.html` voor markup
- `main.js` voor logica
- `styles.scss` voor styling
- Bootstrap layout voor oefen-UI

We verversen dus enkel de inhoud van de `main.js` en eventueel HTML-blokken. Dit geeft structuur en voorkomt chaos.

Template voorbeeld (Smoke Test)

Dit is het stukje dat **bewijst dat je setup werkt**. Als de knop werkt, ben je klaar voor de rest van de cursus. Je krijgt eerst de template van de docent.

Bestand: src/index.html

```
<div class="container py-4">
  <h1 class="h4 mb-3">Vite + Bootstrap werkt </h1>

  <div class="card shadow-sm">
    <div class="card-header bg-primary text-white">
      Test
    </div>
    <div class="card-body">
      <p class="mb-3">Klik op de knop om te testen of alles goed geladen
is.</p>

      <button id="btnTest" class="btn btn-primary">
        Test template
      </button>

      <div id="out" class="alert alert-secondary mt-3 mb-0">
        Output verschijnt hier
      </div>
    </div>
  </div>
</div>
```

Bestand: src/js/main.js

```
import './scss/styles.scss'
import * as bootstrap from 'bootstrap'

document.addEventListener('DOMContentLoaded', () => {
  const btn = document.getElementById('btnTest')
  const out = document.getElementById('out')

  btn.addEventListener('click', () => {
    out.className = 'alert alert-success mt-3 mb-0'
    out.textContent = ' Alles werkt. Klaar voor de oefeningen!'
  })
})
```

Bestand: src/scss/styles.scss

```
@import "bootstrap/scss/bootstrap";

body {
  background-color: #f8f9fa;
}
```

Als je nu `npm run start` doet en klikt op de knop, zie je een groene alert:

Alles werkt. Klaar voor de oefeningen!

Dan is je omgeving 100% klaar.

Mini-opdracht

Voeg onder de testcard een tweede card toe met:

- een tekstinput
- een knop "Toon naam"
- een alert die "Welkom, [naam]" toont

Gebruik Bootstrap classes.

Begeleide oplossing

Toevoeging in index.html (onder de testcard)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Naam test</div>
  <div class="card-body">
    <input id="inpName" class="form-control mb-2" placeholder="Geef je naam in" />
    <button id="btnName" class="btn btn-dark w-100">Toon naam</button>

    <div id="nameOut" class="alert alert-secondary mt-3 mb-0">Nog niets
ingegeven</div>
  </div>
</div>
```

Toevoeging in main.js

```
const btnName = document.getElementById('btnName')
const inpName = document.getElementById('inpName')
const nameOut = document.getElementById('nameOut')

btnName.addEventListener('click', () => {
  const naam = inpName.value.trim()

  if (!naam) {
    nameOut.className = 'alert alert-warning mt-3 mb-0'
    nameOut.textContent = '⚠️ Vul je naam in aub'
    return
  }

  nameOut.className = 'alert alert-success mt-3 mb-0'
  nameOut.textContent = `Welkom, ${naam}!`
})
```

Resultaat

Je hebt nu:

- een werkende Vite + Bootstrap basis
- een vaste oefentemplate
- UI-interactie met DOM
- duidelijke output feedback

We gaan vanaf nu **elke oefening in deze template schrijven**.

Hoofdstuk 2: Functions

Doel van dit hoofdstuk

Na dit hoofdstuk kan je:

- uitleggen wat een functie is
- een functie definiëren en aanroepen
- werken met parameters & return
- pure functies (berekening) scheiden van impure functies (UI)
- input lezen en output tonen met JS
- Bootstrap UI koppelen aan functies

Concept & uitleg

Wat is een functie?

Een functie is **herbruikbare code** die iets doet en (meestal) een resultaat teruggeeft.

Waarom?

- Nooit dubbel werk (**DRY** principe)
- Logica netjes gescheiden
- Helder, leesbaar, uitbreidbaar
- In UI: berekening apart, tonen apart

Structuur

```
function naam(parameter) {  
  return resultaat;  
}
```

naam(waarde)

Belangrijk

Term	Betekenis
Parameter	naam in functie definitie
Argument	waarde die je doorgeeft
Return	wat de functie teruggeeft

Pure vs Impure

Type	Doel
Pure	berekenen, niets aanpassen buiten functie
⚠ Impure	DOM, alert, console.log, data wijzigen

We gebruiken **pure functies** voor **logica**, en **impure functies** voor **UI**.

Voorbeeld in jouw template

We voegen **1 oefenblok** toe met unieke ID's.

src/index.html → onder de vorige testcard voeg je toe:

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">Hoofdstuk 2: Functions</div>
  <div class="card-body">

    <label class="form-label">Wat is je naam?</label>
    <input id="fn_nameInput" class="form-control mb-2" placeholder="vb. Tom">

    <button id="fn_btnGreet" class="btn btn-primary w-100">Groet</button>

    <div id="fn_output" class="alert alert-secondary mt-3 mb-0">
      Resultaat komt hier...
    </div>

  </div>
</div>
```

Opgelet: **fn_ prefix** = “functions chapter”. Zo vermijden we ID-conflicten door heel de cursus.

src/js/main.js → voeg onderaan toe:

```
// -----
// Hoofdstuk 2: Functions
// -----

// Pure function (berekening)
function maakGroet(naam) {
  return `Hallo ${naam}!`;
}

// Impure function (UI)
function handleGroetClick() {
  const naamInput = document.getElementById('fn_nameInput');
  const output = document.getElementById('fn_output');

  const naam = naamInput.value.trim();

  if (!naam) {
    output.className = "alert alert-warning mt-3 mb-0";
    output.textContent = "⚠ Geef een naam in!";
  }
}
```

```

    return;
  }

  const boodschap = maakGroet(naam);

  output.className = "alert alert-success mt-3 mb-0";
  output.textContent = boodschap;
}

// Event koppelen
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById('fn_btnGreet')
    ?.addEventListener("click", handleGroetClick);
});

```

Expected behaviour

- Lege input → gele waarschuwing
- Geldige naam → groene groet

Dit lijkt simpel, maar vormt het fundament voor **ELK volgend hoofdstuk**.

Mini-opdracht

Maak functie `berekenVierkant(getal)`:

Input	Output
5	"5 ² = 25"
10	"10 ² = 100"

UI:

- input veld (nummer)
- knop "Bereken vierkant"
- Bootstrap alert onderaan

Zelfde ID-stijl:

```

sq_input
sq_btn
sq_output

```

Begeleide oplossing

HTML (onder vorige card)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht: Vierkant
  berekenen</div>
  <div class="card-body">

    <input id="sq_input" type="number" class="form-control mb-2"
placeholder="vb. 5">
    <button id="sq_btn" class="btn btn-dark w-100">Bereken vierkant</button>

    <div id="sq_output" class="alert alert-secondary mt-3 mb-0">
      Resultaat komt hier...
    </div>

  </div>
</div>
```

JS

```
// Pure berekening
function berekenVierkant(getal) {
  return getal * getal;
}

// UI handler
function handleVierkant() {
  const inp = document.getElementById('sq_input');
  const out = document.getElementById('sq_output');

  const waarde = Number(inp.value);

  if (!waarde) {
    out.className = "alert alert-warning mt-3 mb-0";
    out.textContent = "⚠ Geef een getal in!";
    return;
  }

  const resultaat = berekenVierkant(waarde);

  out.className = "alert alert-success mt-3 mb-0";
  out.textContent = `${waarde}2 = ${resultaat}`;
}

// Event koppelen
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById('sq_btn')
    ?.addEventListener("click", handleVierkant);
});
```

Hoofdstuk 3: Function Scoping

Doel van dit hoofdstuk

Na dit hoofdstuk kan je:

- uitleggen wat **scope** betekent
- het verschil begrijpen tussen **function scope** en **block scope**
- lokale vs globale variabelen herkennen
- nested functions gebruiken
- scoping-fouten herkennen zoals `ReferenceError: x is not defined`

Concept & uitleg

Wat is scope?

Scope bepaalt **waar** een variabele toegankelijk is.

Scope type	Betekenis
Global scope	overal zichtbaar
Function scope	enkel binnen een functie zichtbaar
Block scope (<code>let</code> , <code>const</code>)	enkel binnen <code>{ }</code> zichtbaar
Lexical scope	inner functions kunnen outer data gebruiken

ES6 sleutelregel

Gebruik `let` en `const` ✗ gebruik geen `var` meer ondanks dat dit altijd zal blijven werken (oude function-scoped variabelen)

Klassieke bug

```
{  
  let x = 5;  
}  
console.log(x); // ✗ ReferenceError
```

Nested functions

```
function outer() {  
  const naam = "Tom";  
  
  function inner() {  
    console.log(naam); // werkt  
  }  
  
  inner();  
}
```

Binnenste ziet buitenste. Buitenste ziet binnenste **niet**.

Denkbeeld

Stel je voor:

- Je code is een huis.
- Een functie is een **kamer**.
- Wat in de kamer ligt, kan je daar gebruiken.
- Maar je kan niet door de muur kijken naar binnen.

Inner functions = een klein kamertje in de kamer.

Oefening: Function Scope Demo

src/index.html → voeg dit oefenblok toe:

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">Hoofdstuk 3: Function
  Scoping</div>
  <div class="card-body">

    <button id="sc_btn" class="btn btn-info w-100 mb-2">Voer scope demo
    uit</button>

    <div id="sc_output" class="alert alert-secondary mb-0">
      Klik op de knop om scope te testen...
    </div>

  </div>
</div>
```

src/js/main.js → voeg onderaan toe:

```
// -----
// Hoofdstuk 3: Function Scoping
// -----

function toonBericht() {
  const binnen = "Ik leef binnen de functie 🐼";

  function inner() {
    return `Inner ziet: ${binnen}`;
  }

  return inner();
}

function handleScope() {
  const output = document.getElementById("sc_output");

  // inner sees outer
  const bericht = toonBericht();

  output.className = "alert alert-success mb-0";
  output.textContent = bericht;
}
```

```

try {
  // ✗ buiten de functie proberen de variabele te gebruiken
  console.log(binnen);
} catch (err) {
  console.warn("binnen is niet zichtbaar buiten de functie");
}
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("sc_btn")
    ?.addEventListener("click", handleScope);
});

```

Expected behaviour

Klik op de knop → groene alert:

Inner ziet: Ik leef binnen de functie 🗝️

Console:

binnen is niet zichtbaar buiten de functie

Mini-opdracht

Maak:

- functie geheimBericht()
- lokale variabele secret = "Code unlocked 🗝️"
- nested function leesSecret()
- return dat inner result
- UI met knop + output

Gebruik ID's:

sc2_btn
sc2_output

Hint: zelfde patroon als boven, andere tekst.

Begeleide oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-warning">Mini-opdracht: Geheim bericht</div>
  <div class="card-body">
    <button id="sc2_btn" class="btn btn-warning w-100 mb-2">Toon
geheim</button>
    <div id="sc2_output" class="alert alert-secondary mb-0">
      Wacht op geheim...
    </div>
  </div>
</div>
```

JS

```
function geheimBericht() {
  const secret = "Code unlocked 🔑";

  function leesSecret() {
    return secret;
  }

  return leesSecret();
}

function handleSecret() {
  const out = document.getElementById("sc2_output");
  out.className = "alert alert-success mb-0";
  out.textContent = geheimBericht();
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("sc2_btn")
    ?.addEventListener("click", handleSecret);
});
```

Wat je nu beheerst

- Function scope
- Block scope
- Lexical scope
- Inner vs outer toegang
- Variable accessibility errors herkennen

Dit is superbelangrijk voor:

- Callbacks
- Modules
- DOM events
- Promises / async
- API calls later

Hoofdstuk 4: Named Functions

Doel van dit hoofdstuk

Na deze module kan je:

- uitleggen wat een **named function** is
- function declarations schrijven
- named functions koppelen aan knoppen
- het verschil begrijpen tussen **pure** en **impure** functies
- foutmeldingen beter lezen dankzij functienamen
- input → verwerking → output met duidelijke code

Theorie: Named Functions

Wat is een named function?

Een functie met een **duidelijke naam**:

```
function zegHallo() {  
  return "Hallo!";  
}
```

Waarom belangrijk?

Reden	Waarom
Lesbaarheid	Je ziet in één oogopslag wat code doet
Debugging	Foutmeldingen tonen functienaam
Structuur	Bouwblokken die makkelijk herbruikbaar zijn
Teamwerk	Iedereen begrijpt het sneller

Named vs Anonymous

Slecht (moeilijk debuggen)

```
button.addEventListener("click", function() {  
  ...  
});
```

Goed (duidelijk & traceerbaar)

```
button.addEventListener("click", toonKleur);
```

Demo: Kleur tonen met named functions

We bouwen:

- inputveld
- knop
- tekstvak
- gekleurde box (visuele feedback)

src/index.html (+ blok onder vorige card)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-secondary text-white">
    Hoofdstuk 4: Named Functions
  </div>
  <div class="card-body">

    <label class="form-label">Typ een primaire kleur (rood of blauw):</label>
    <input id="nf_input" class="form-control mb-2" placeholder="bv. rood">

    <button id="nf_btn" class="btn btn-secondary w-100 mb-2">Toon
kleur</button>

    <div id="nf_text" class="alert alert-secondary mb-2">
      Wacht op input...
    </div>

    <div id="nf_box"
      class="d-flex justify-content-center align-items-center fw-bold"
      style="height: 70px; border-radius: 6px; border: 1px solid #ccc;
background:#f8f9fa;">
      Geen kleur
    </div>

  </div>
</div>
```

src/js/main.js (+ onderaan toevoegen)

```
// -----
// Hoofdstuk 4: Named Functions
// -----

// Pure function
function bepaalKleurInfo(kleur) {
  const input = kleur.toLowerCase();

  if (input === "rood") return { text: "Je koos rood", color: "red" };
  if (input === "blauw") return { text: "Je koos blauw", color: "blue" };

  return { text: "Onbekende kleur", color: null };
}

// Impure function (DOM)
function toonKleur() {
  const inp = document.getElementById("nf_input");
  const out = document.getElementById("nf_text");
  const box = document.getElementById("nf_box");

  const waarde = inp.value.trim();

  if (!waarde) {
    out.className = "alert alert-warning mb-2";
    out.textContent = "⚠ Geef een kleur in";
  }
}
```

```

    box.style.background = "#f8f9fa";
    box.textContent = "Geen kleur";
    return;
}

const resultaat = bepaalKleurInfo(waarde);

if (!resultaat.color) {
    out.className = "alert alert-danger mb-2";
    out.textContent = resultaat.text;
    box.style.background = "#f8f9fa";
    box.textContent = "Onbekend";
    return;
}

out.className = "alert alert-success mb-2";
out.textContent = resultaat.text;
box.style.background = resultaat.color;
box.textContent = resultaat.color.toUpperCase();
}

// Event koppeling
document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("nf_btn")
        ?.addEventListener("click", toonKleur);
});

```

Wat zagen we?

- Functie **met naam** is beter traceerbaar
- Pure functie berekent
- Impure functie werkt met UI
- Input → verwerking → output + visuele feedback

Mini-opdracht

Maak een **fruit-kiezer** met emoji.

Vereisten

Input	Tekst	Emoji
appel	"Je koos appel"	🍏
banaan	"Je koos banana"	🍌
anders	waarschuwing	?

UI-elementen

- Input veld
- Knop
- Tekstvak (alert)
- Blok met emoji

Verplichte ID's

fr_input
fr_btn
fr_text
fr_box

Tip

Pure function retourneert:

```
{ text: "Je koos appel", emoji: "🍏" }
```

Begeleide oplossing mini-opdracht

HTML

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-warning">Mini-opdracht: Fruit kiezer</div>  
  <div class="card-body">  
  
    <input id="fr_input" class="form-control mb-2" placeholder="bv. appel">  
    <button id="fr_btn" class="btn btn-warning w-100 mb-2">Kies  
fruit</button>  
  
    <div id="fr_text" class="alert alert-secondary mb-2">Wacht op  
input...</div>  
  
    <div id="fr_box"  
      class="d-flex justify-content-center align-items-center fw-bold"  
      style="height:70px; border:1px solid #ccc; border-radius:6px;  
background:#f8f9fa;">  
      ?  
    </div>  
  
  </div>  
</div>
```

JS

```
function bepaalFruit(naam) {  
  const f = naam.toLowerCase();  
  
  if (f === "appel") return { text: "Je koos appel", emoji: "🍏" };  
  if (f === "banaan") return { text: "Je koos banaan", emoji: "🍌" };  
  
  return { text: "Onbekend fruit", emoji: "?" };  
}  
  
function toonFruit() {  
  const inp = document.getElementById("fr_input");  
  const txt = document.getElementById("fr_text");  
  const box = document.getElementById("fr_box");  
  
  const waarde = inp.value.trim();
```

```

if (!waarde) {
  txt.className = "alert alert-warning mb-2";
  txt.textContent = "⚠️ Vul een fruitsoort in!";
  box.textContent = "?";
  return;
}

const res = bepaalFruit(waarde);

txt.className = "alert alert-success mb-2";
txt.textContent = res.text;
box.textContent = res.emoji;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("fr_btn")
    ?.addEventListener("click", toonFruit);
});

```

Samenvatting

Concept	Belangrijk
Named functions	leesbaar, debugbaar
Pure functions	berekenen, géén DOM
Impure	DOM manipulatie
Mapping	user input naar UI resultaat
UX	visuele feedback is cruciaal

Hoofdstuk 5: Functions & Strings in ES6 (Template Literals)

Leerdoelen

Na dit hoofdstuk kan je:

- ES6 **template literals** gebruiken (``)`)
- variabelen en functies verwerken in strings
- multiline strings schrijven
- dynamische tekst genereren in UI
- string-interpolatie gebruiken in JS functies

Theorie: Template Literals

Oude manier (slecht leesbaar)

```

const naam = "Tom";
const zin = "Hallo, " + naam + "! Welkom.";

```

Moderne ES6 manier

```

const naam = "Tom";
const zin = `Hallo, ${naam}! Welkom.`;

```

Voordelen

Voordeel	Waarom
Leesbaar	natuurlijke zinnen
Variabelen in string	<code>\${variabele}</code>
Multiline strings	geen <code>\n</code> nodig
Perfect voor HTML	dynamische UI content

Multiline voorbeeld

```
const blokje = `
Dit is een titel
En dit is een tweede lijn
Het leest heerlijk
`;
```

Real-life analogie

Template literals zijn **post-its met invulvelden**:

“Welkom, **{naam}**! Je score is **{score}**.”

In JS:

```
` ${naam} ${score} `
```

Demo: Welkomstbericht generator

We bouwen:

- input veld voor naam
- input veld voor leeftijd
- knop
- output met template literal

index.html (plaats onder vorige card)

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">
    Hoofdstuk 5: Template Literals
  </div>
  <div class="card-body">

    <label class="form-label">Naam:</label>
    <input id="tl_name" class="form-control mb-2" placeholder="bv. Tom">

    <label class="form-label">Leeftijd:</label>
    <input id="tl_age" type="number" class="form-control mb-2"
placeholder="bv. 25">

    <button id="tl_btn" class="btn btn-info w-100 mb-2">Maak
welkomstbericht</button>

    <div id="tl_output" class="alert alert-secondary mb-0">
      Wacht op input...
```

```

    </div>

</div>
</div>

main.js (onderaan toevoegen)
// -----
// Hoofdstuk 5: Template Literals
// -----

// Pure function
function maakWelkomstZin(naam, leeftijd) {
  return `Welkom ${naam}! Jij bent ${leeftijd} jaar oud.`;
}

// UI handler
function toonWelkomstZin() {
  const naam = document.getElementById("tl_name").value.trim();
  const leeftijd = document.getElementById("tl_age").value.trim();
  const out = document.getElementById("tl_output");

  if (!naam || !leeftijd) {
    out.className = "alert alert-warning mb-0";
    out.textContent = `⚠️ Vul naam en leeftijd in`;
    return;
  }

  const tekst = maakWelkomstZin(naam, leeftijd);

  out.className = "alert alert-success mb-0";
  out.textContent = tekst;
}

// Event
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("tl_btn")
    ?.addEventListener("click", toonWelkomstZin);
});

```

Output type:

Welkom Tom! Jij bent 25 jaar oud.

Mini-opdracht

Maak een **profielkaart generator**.

Inputvelden

- naam
- stad
- hobby

Output (Template Literal)

Voorbeeld:

Hey, ik ben Sara uit Gent.
Mijn hobby is schilderen.
Leuk je te ontmoeten!

Gebruik:

- 3 inputvelden
- 1 knop
- 1 output <div>
- Template literal met minimaal **twee regels tekst**
- Validatie (geen lege waarden)

Verplichte ID's

pf_name
pf_city
pf_hobby
pf_btn
pf_out

Mini-opdracht begeleide oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht:
  Profielkaart</div>
  <div class="card-body">

    <input id="pf_name" class="form-control mb-2" placeholder="Naam">
    <input id="pf_city" class="form-control mb-2" placeholder="Stad">
    <input id="pf_hobby" class="form-control mb-2" placeholder="Hobby">

    <button id="pf_btn" class="btn btn-dark w-100 mb-2">Maak profiel</button>

    <div id="pf_out" class="alert alert-secondary mb-0">
      Wacht op input...
    </div>

  </div>
</div>
```

JS

```
function maakProfielZin(naam, stad, hobby) {
  return `
  Hey, ik ben ${naam} uit ${stad}.
  Mijn hobby is ${hobby}.
  Leuk je te ontmoeten!
  `;
}

function toonProfiel() {
  const naam = document.getElementById("pf_name").value.trim();
  const stad = document.getElementById("pf_city").value.trim();
  const hobby = document.getElementById("pf_hobby").value.trim();
  const out = document.getElementById("pf_out");

  if (!naam || !stad || !hobby) {
    out.className = "alert alert-warning mb-0";
    out.textContent = `⚠️ Vul alle velden in`;
    return;
  }

  const tekst = maakProfielZin(naam, stad, hobby);

  out.className = "alert alert-success mb-0";
  out.textContent = tekst;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("pf_btn")
    ?.addEventListener("click", toonProfiel);
});
```

Samenvatting

Concept	Uitleg
Template literals	`Hallo \${naam}` ``
String interpolation	variabelen in tekst
Multiline strings	tekstblokken zonder
Pure fn	maakt tekst
UI fn	toont tekst in alert

Hoofdstuk 6: ES6 Arrow Functions (=>)

Leerdoelen

Na dit hoofdstuk kan je:

- een klassieke functie omschrijven naar een **arrow function**
- arrow function syntax herkennen en toepassen
- **implicit return** gebruiken
- parameters correct verwerken
- arrow vs gewone functies begrijpen (basis)
- arrow functions in UI handlers toepassen

We focussen eerst op **syntaxis & gebruik**. Het verschil in this leren we later bij OOP.

Theorie: Arrow Functions

Arrow functions zijn **kortere ES6 functies**:

Klassiek

```
function optellen(a, b) {  
  return a + b;  
}
```

Arrow

```
const optellen = (a, b) => {  
  return a + b;  
};
```

Arrow met implicit return 📦 (één regel!)

```
const optellen = (a, b) => a + b;
```

Enkele parameters (haakjes optioneel)

```
const vierkant = x => x * x;
```

Geen parameters

```
const hallo = () => "Hallo!";
```

Waarom gebruiken?

Voordeel	Waarom
Kort	minder typwerk
Duidelijk	super voor kleine functies
Modern	hoort bij ES6 mindset
Perfect voor callbacks	events, array methods straks

Vergelijking

Klassiek	Arrow
function(x){ return x+1 }	x => x + 1
function(){}	() => {}
function naam(){}	const naam = () => {}

Let op

Arrow functions hebben **geen eigen this**. Daarover later bij OOP.

Oefening: Bereken + Toon (arrow style)

We bouwen:

- 2 inputvelden
- optellen met arrow function
- resultaat tonen met Bootstrap alert

index.html blok

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-success text-white">
    Hoofdstuk 6: Arrow Functions
  </div>
  <div class="card-body">

    <label class="form-label">Getal 1:</label>
    <input id="af_num1" type="number" class="form-control mb-2"
placeholder="vb. 5">

    <label class="form-label">Getal 2:</label>
    <input id="af_num2" type="number" class="form-control mb-2"
placeholder="vb. 7">

    <button id="af_btn" class="btn btn-success w-100 mb-2">Tel op</button>

    <div id="af_out" class="alert alert-secondary mb-0">
      Wacht op input...
    </div>

  </div>
</div>
```

main.js blok

```
// -----
// Hoofdstuk 6: Arrow Functions
// -----

// Pure arrow function
const telOp = (a, b) => a + b;

// UI handler (arrow + block body)
const verwerkSom = () => {
  const n1 = Number(document.getElementById("af_num1").value);
  const n2 = Number(document.getElementById("af_num2").value);
  const out = document.getElementById("af_out");

  if (!n1 || !n2) {
    out.className = "alert alert-warning mb-0";
    out.textContent = `⚠️ Vul twee getallen in`;
  }
}
```

```

    return;
}

const resultaat = telOp(n1, n2);

out.className = "alert alert-success mb-0";
out.textContent = `Resultaat: ${n1} + ${n2} = ${resultaat}`;
};

// Event
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("af_btn")
    ?.addEventListener("click", verwerkSom);
});

```

Wat zag je?

- pure arrow function: `const telOp = (a, b) => a + b`
- arrow function als event handler
- implicit return
- input → berekening → output

Mini-opdracht

Maak een arrow function die checkt of een getal even is.

Verwacht gedrag

Input	Output
4	"4 is even"
7	✗ "7 is niet even"

IDs te gebruiken

ev_input
ev_btn
ev_out

hints

Arrow function:

```
const isEven = num => num % 2 === 0;
```

Oplossing mini-opdracht

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht: Even
  checker</div>
  <div class="card-body">

    <input id="ev_input" type="number" class="form-control mb-2"
    placeholder="vb. 6">
    <button id="ev_btn" class="btn btn-dark w-100 mb-2">Controleer</button>

    <div id="ev_out" class="alert alert-secondary mb-0">
      Wacht op input...
    </div>

  </div>
</div>
```

JS

```
const isEven = num => num % 2 === 0;

const checkEven = () => {
  const inp = Number(document.getElementById("ev_input").value);
  const out = document.getElementById("ev_out");

  if (!inp) {
    out.className = "alert alert-warning mb-0";
    out.textContent = `⚠️ Vul een getal in`;
    return;
  }

  const resultaat = isEven(inp)
    ? `${inp} is EVEN`
    : `${inp} is ONEVEN ❌`;

  out.className = "alert alert-success mb-0";
  out.textContent = resultaat;
};

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("ev_btn")
    ?.addEventListener("click", checkEven);
});
```

Samenvatting

Je kent nu	Wat het betekent
Arrow syntax	const fn = () => {...}
Implicit return	x => x * 2
Event handler met arrow	btn.addEventListener("click", ()=>{})
Separatie van logica en UI	clean & schaalbaar

Hoofdstuk 7: ES6 Arrays

Leerdoelen

Na dit hoofdstuk kan je:

- een array definiëren in ES6
- items toevoegen, uitlezen en tellen
- `.push()` en `.length` gebruiken
- input opslaan in een array
- een lijst tonen in de UI via JavaScript
- basis denken richting datastructuren

Theorie: Arrays

Wat is een array?

Een **lijst** met meerdere waarden in één variabele.

```
const namen = ["Tom", "Sara", "Liam"];
```

Waarom arrays?

Voordeel	Waarom
Bewaart meerdere waarden	“groepeert data”
Behoud volgorde	belangrijk voor logica
Makkelijk uitbreiden	<code>.push(...)</code>
Basis voor echte databanken	denken in collecties

Basis handelingen

Doel	Code	Uitleg
Item ophalen	<code>namen[0]</code>	eerste item
Aantal items	<code>namen.length</code>	lengte van lijst
Item toevoegen	<code>namen.push("Jan")</code>	voeg achteraan toe

Vergelijking in real life

Een array is een **rij stoelen** in een klas. Iedere stoel = een plek voor data.

Stoel 0 = eerste Stoel 1 = tweede Stoel 2 = derde ...

Programmeren is vaak “data + acties op data”. Arrays zijn je rugzak.

Demo: gastenlijst (array + UI)

We bouwen:

- input veld "naam"
- knop "voeg toe"
- array bewaart namen
- lijstje toont alle namen live

index.html blok

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">
    Hoofdstuk 7: Arrays
  </div>
  <div class="card-body">

    <label class="form-label">Voeg een naam toe:</label>
    <input id="arr_name" class="form-control mb-2" placeholder="bv. Lisa">

    <button id="arr_btn" class="btn btn-primary w-100 mb-2">Voeg toe</button>

    <div class="alert alert-info mb-2">
      Totaal namen: <span id="arr_count">0</span>
    </div>

    <ul id="arr_list" class="list-group"></ul>

  </div>
</div>
```

main.js blok

```
// -----
// Hoofdstuk 7: Arrays
// -----

// Lege array om namen op te slaan
const namen = [];

// Pure functie -> maakt lijst HTML
function maakLijstHTML(items) {
  return items.map(item => `<li class="list-group-item">${item}</li>`).join("");
}

// UI handler
function voegNaamToe() {
  const inp = document.getElementById("arr_name");
  const lijst = document.getElementById("arr_list");
  const count = document.getElementById("arr_count");

  const naam = inp.value.trim();

  if (!naam) {
```

```

    alert("⚠ Geef een naam in!");
    return;
}

// voeg toe aan array
namen.push(naam);

// UI updaten
lijst.innerHTML = maakLijstHTML(namen);
count.textContent = namen.length;

// veld leegmaken
inp.value = "";
}

// Event Listener
document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("arr_btn")
        ?.addEventListener("click", voegNaamToe);
});

```

Je hebt nu een **dynamische gastenlijst**. Dit is al een mini-database.

Mini-opdracht

Maak een **takenlijst** (todo list).

Vereisten

Inputveld: taak Knop: "Voeg taak toe" Lijst toont taken Tel taken in badge

Verplichte ID's

```

todo_input
todo_btn
todo_list
todo_count

```

Extra uitdaging

Voeg aan elk item toe (alleen tekst, nog geen echte delete-knop).

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht: Takenlijst</div>
  <div class="card-body">

    <input id="todo_input" class="form-control mb-2" placeholder="Nieuwe
taak">
    <button id="todo_btn" class="btn btn-dark w-100 mb-2">Voeg toe</button>

    <div class="alert alert-secondary mb-2">
      Taken: <span id="todo_count">0</span>
    </div>

    <ul id="todo_list" class="list-group"></ul>

  </div>
</div>
```

JS

```
const taken = [];

function maakTodoHTML(items) {
  return items.map(item => `<li class="list-group-item">
${item}</li>`).join("");
}

function voegTaakToe() {
  const inp = document.getElementById("todo_input");
  const lijst = document.getElementById("todo_list");
  const count = document.getElementById("todo_count");

  const taak = inp.value.trim();

  if (!taak) {
    alert("⚠️ Vul een taak in!");
    return;
  }

  taken.push(taak);
  lijst.innerHTML = maakTodoHTML(taken);
  count.textContent = taken.length;
  inp.value = "";
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("todo_btn")
    ?.addEventListener("click", voegTaakToe);
});
```

.

Samenvatting

Je hebt geleerd	Betekenis
<code>[]</code>	array maken
<code>.push()</code>	waarde toevoegen
<code>.length</code>	aantal items ophalen
<code>map()</code>	array omzetten naar HTML
State + UI	basis van moderne apps

Dit is de fundering voor:

- todo apps
- databanken
- API's & JSON
- frameworks zoals Vue & React

Hoofdstuk 8: DOM Nodes & Root Nodes

Leerdoelen

Na dit hoofdstuk kan je:

- uitleggen wat een **DOM-node** is
- **root nodes** herkennen (`document`, `document.documentElement`, `document.body`)
- HTML als **boomstructuur** zien
- nodes inspecteren in de browser
- root nodes aanspreken in JavaScript
- tekst in de pagina zetten via root-node selectie

Theorie

Wat is de DOM?

DOM = Document Object Model

De DOM is een **boom van nodes** die jouw HTML in JavaScript voorstelt.

Browser vertaalt HTML → boomstructuur:

```
document
├── html
│   ├── head
│   └── body
```

Wat is een Node?

Een **bouwblok** van de DOM.

Voorbeelden:

HTML	Node type
<html>	Element node
<body>	Element node
Tekst tussen tags	Text node
Commentaar	Comment node

Root nodes (basis van alles)

Node	Betekenis
document	startpunt van de DOM
document.documentElement	<html> element
document.body	<body> element

Visualisatie in devtools

Rechtsklik → Inspect → Elements

Hier zie je de DOM live. Dit is hoe JS je website **ziet**.

Denkbeeld

HTML = **huisplan** DOM = **het echte gebouw in 3D** waarin je rondwandelt Nodes = **kamers, muren, deuren**

Demo: tekst zetten via root nodes

We maken een card die:

- tekst schrijft naar <body>
- achtergrond verandert
- toont welke root node we selecteren

index.html blok

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">
    Hoofdstuk 8: DOM Root Nodes
  </div>
  <div class="card-body">

    <button id="rn_btnMessage" class="btn btn-info w-100 mb-2">Zet bericht in
body</button>
    <button id="rn_btnColor" class="btn btn-dark w-100 mb-2">Verander
achtergrond</button>

    <div id="rn_output" class="alert alert-secondary mb-0">
      Wacht op actie...
    </div>
```

```

    </div>
  </div>

  main.js blok
  // -----
  // Hoofdstuk 8: Root Nodes
  // -----

  function plaatsBerichtInBody() {
    // selecteer de body
    const bodyNode = document.body;

    // maak een nieuw element
    const p = document.createElement("p");
    p.textContent = "Bericht toegevoegd via root node! ";
    p.className = "text-center mt-2 text-success fw-bold";

    // voeg toe aan body
    bodyNode.appendChild(p);
  }

  function veranderAchtergrond() {
    const htmlNode = document.documentElement; // <html>

    // lichte highlight
    htmlNode.style.background = "#e6f3ff";
  }

  function toonFeedback(tekst) {
    const out = document.getElementById("rn_output");
    out.className = "alert alert-success mb-0";
    out.textContent = tekst;
  }

  // Event Listeners
  document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("rn_btnMessage")
      ?.addEventListener("click", () => {
        plaatsBerichtInBody();
        toonFeedback("Bericht toegevoegd aan body ");
      });

    document.getElementById("rn_btnColor")
      ?.addEventListener("click", () => {
        veranderAchtergrond();
        toonFeedback("Achtergrond aangepast via <html> node 🌀");
      });
  });

```

Wat gebeurde hier?

- We selecteerden root nodes
- We creëerden een nieuw element via JS
- We voegden dat toe aan <body>
- We veranderden stijl via <html>

Jij kunt nu de **DOM-boom manipuleren vanaf de wortel**.

Mini-opdracht

Maak een knop die:

- <body> een donker thema geeft
- tekst toont: "Dark mode geactiveerd 🌙"
- tweede knop maakt het weer licht

Verplichte ID's:

dm_on
dm_off
dm_status

Effecten

Actie	Effect
Dark mode	document.body.style.background = "#222", text wit
Light mode	reset naar wit

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-dark text-white">Mini-opdracht: Dark Mode</div>  
  <div class="card-body">  
  
    <button id="dm_on" class="btn btn-dark w-100 mb-2">Dark mode 🌙</button>  
    <button id="dm_off" class="btn btn-light w-100 mb-2">Light mode  
    🌞</button>  
  
    <div id="dm_status" class="alert alert-secondary mb-0">Wacht op  
actie...</div>  
  
  </div>  
</div>
```

JS

```
function activeerDarkMode() {  
  document.body.style.background = "#222";  
  document.body.style.color = "#fff";  
}
```

```
function activeerLightMode() {  
  document.body.style.background = "#fff";  
}
```

```

    document.body.style.color = "#000";
}

function toonStatus(msg) {
    const out = document.getElementById("dm_status");
    out.className = "alert alert-success mb-0";
    out.textContent = msg;
}

document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("dm_on")
        ?.addEventListener("click", () => {
            activeerDarkMode();
            toonStatus("Dark mode actief 🌙");
        });

    document.getElementById("dm_off")
        ?.addEventListener("click", () => {
            activeerLightMode();
            toonStatus("Light mode actief ☀️");
        });
});

```

Samenvatting

Concept	Wat je nu weet
DOM	Boomstructuur van pagina
Node	elk element in boom
Root nodes	document, html, body
Manipulatie	DOM aanpassen via JS
Append	elementen dynamisch toevoegen

Jij kunt nu **vanaf de wortel de pagina beïnvloeden**. Dat is key voor alles wat volgt.

Volgend hoofdstuk

Hoofdstuk 9: JavaScript Output Methods

We gaan:

- verschillende manieren ontdekken om output te tonen
- DOM vs console vs.alert
- waar je wat gebruikt in echte projecten

Als je klaar bent:

doorgaan Geweldig. We bouwen verder aan jouw JS-fundament.

Hoofdstuk 9: JavaScript Output Methodes

Leerdoelen

Na dit hoofdstuk kan je:

- verschillende manieren gebruiken om informatie te tonen in JavaScript
- begrijpen wanneer je welke output-methode gebruikt
- console debuggen, alert tonen, HTML manipuleren
- Bootstrap alerts gebruiken voor nette UI-meldingen
- DOM output methoden vergelijken (**innerText** vs **textContent** vs **innerHTML** volgt in apart hoofdstuk)

Theorie: Output Methodes

1) Console

Voor developers, debugging en testen

```
console.log("Dit is een log");  
console.warn("Dit is een waarschuwing");  
console.error("Dit is een fout");
```

Waarom gebruiken? Debug info Fouten lezen Data bekijken (arrays, objects)

2) Alert

Popup waarschuwing voor gebruiker (zeldzaam in echte apps)

```
alert("Hallo gebruiker!");
```

Waarom voorzichtig mee zijn? ⚠️ Blokkeert pagina ⚠️ Niet mooi ➡️ Enkel voor snelle testjes of dringende meldingen

3) Tekst op pagina

Mooiste manier voor eindgebruikers

```
document.getElementById("id").textContent = "Hallo!";
```

Gebruiken voor: Meldingen UI feedback Output van berekeningen

We gebruiken **Bootstrap alerts** voor professioneel resultaat.

Demo oefening: 3 outputmethoden

Wat we bouwen

- 3 knoppen: Console, Alert, UI
- Output naar console, popup en HTML card
- Zichtbaar leerresultaat

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">
    Hoofdstuk 9: Output Methodes
  </div>
  <div class="card-body">

    <button id="om_console" class="btn btn-outline-primary w-100 mb-2">
      Console Log
    </button>

    <button id="om_alert" class="btn btn-outline-warning w-100 mb-2">
      Alert Meldingsvenster
    </button>

    <button id="om_ui" class="btn btn-outline-success w-100 mb-2">
      Toon in pagina
    </button>

    <div id="om_output" class="alert alert-secondary mb-0">
      Output wordt hier zichtbaar...
    </div>

  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 9: Output Methodes
// -----

// Pure tekstfunctie
function maakBericht() {
  return "Hello output wereld! 🌍";
}

// Impure UI output
function toonInUI(msg) {
  const out = document.getElementById("om_output");
  out.className = "alert alert-success mb-0";
  out.textContent = msg;
}

// Event handlers
function logNaarConsole() {
  console.log(maakBericht());
}
```

```

}

function toonAlert() {
  alert(maakBericht());
}

function toonOutputUI() {
  toonInUI(maakBericht());
}

// Event Listeners
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("om_console")?.addEventListener("click",
logNaarConsole);
  document.getElementById("om_alert")?.addEventListener("click", toonAlert);
  document.getElementById("om_ui")?.addEventListener("click", toonOutputUI);
});

```

Console Alert UI weergave Functie-ontkoppeling (clean code)

Mini-opdracht

Maak een blok dat:

- 2 inputvelden heeft: voornaam en achternaam
- knop: "Toon resultaten"
- output toont op 3 manieren:

Methode	Wat
console	"Voornaam: Tom, Achternaam: Vanhoutte"
alert	"Hallo Tom Vanhoutte!"
pagina	Bootstrap alert met volledige naam

Verplichte ID's

```

on_first
on_last
on_btn
on_out

```

Hint

Gebruik template literals:

```

`Hallo ${voornaam} ${achternaam}!`

```


Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht: Output All</div>
  <div class="card-body">

    <input id="on_first" class="form-control mb-2" placeholder="Voornaam">
    <input id="on_last" class="form-control mb-2" placeholder="Achternaam">

    <button id="on_btn" class="btn btn-dark w-100 mb-2">Toon
resultaten</button>

    <div id="on_out" class="alert alert-secondary mb-0">Wacht op input...</div>

  </div>
</div>
```

JS

```
function toonAllOutputs() {
  const first = document.getElementById("on_first").value.trim();
  const last = document.getElementById("on_last").value.trim();
  const out = document.getElementById("on_out");

  if (!first || !last) {
    out.className = "alert alert-warning mb-0";
    out.textContent = "⚠️ Vul naam en achternaam in";
    return;
  }

  const line = `Voornaam: ${first}, Achternaam: ${last}`;
  const hello = `Hallo ${first} ${last}!`;

  console.log(line);
  alert(hello);

  out.className = "alert alert-success mb-0";
  out.textContent = hello;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("on_btn")
    ?.addEventListener("click", toonAllOutputs);
});
```

Samenvatting

Methode	Gebruik
console.log	Debugging / leren
alert()	Snel testen
DOM output	Echte user feedback

Je kunt nu **verschillende output kanalen** gebruiken en weet **waarom**.

Hoofdstuk 10: document.write() en document.writeln()

Leerdoelen

Na dit hoofdstuk kan je:

- uitleggen wat document.write() doet
- het verschil kennen tussen .write() en .writeln()
- begrijpen waarom deze functies **niet** meer gebruikt worden in moderne webapps
- de moderne alternatieven toepassen

Theorie

Wat doet document.write()?

Schrijft **rechtstreeks** tekst/HTML naar de pagina **tijdens het laden**.

```
document.write("Hallo wereld");
```

Wat doet document.writeln()?

Hetzelfde, maar voegt een **lijnbreuk** toe.

```
document.writeln("Hallo");  
document.writeln("wereld");
```

Waarom gebruiken we dit bijna nooit meer?

Probleem	Uitleg
Overschrijft pagina	Alles kan verdwijnen als je het na load gebruikt
Geen controle	Plakt HTML direct op document-stream
Niet modern	Geen componenten, geen dynamiek
Niet toegankelijk	Breekt makkelijk layouts

Waarvoor bestaat het nog?

Zeer oude tutorials Legacy systemen leren begrijpen Debuggen in *heel oude* scripts

Moderne alternatieven

Gebruik:

- textContent
- innerHTML
- DOM methods (appendChild, createElement)
- Bootstrap alerts

We gebruiken **document.write één keer**, puur educatief.

Demo: Waarom je het niet gebruikt

index.html blok

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-danger text-white">
    Hoofdstuk 10: document.write() (niet meer gebruiken)
  </div>
  <div class="card-body">

    <button id="dw_btn" class="btn btn-danger w-100 mb-2">
      document.write uitvoeren
    </button>

    <div id="dw_msg" class="alert alert-secondary mb-0">
      Druk op de knop en let op de pagina...
    </div>

  </div>
</div>
```

main.js blok

```
// -----
// Hoofdstuk 10: document.write()
// -----

function demonstreerWrite() {
  // Waarschuwing tonen
  alert("Let op: de pagina zal overschreven worden!");

  // Dit overschrijft alle HTML
  document.write("<h1>De hele pagina is vervangen 😊</h1>");
  document.write("<p>Daarom gebruiken we dit niet meer.</p>");
  document.writeln("<p>Gebruik DOM methods en Bootstrap.</p>");
}

// Event
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("dw_btn")
    ?.addEventListener("click", demonstreerWrite);
});
```

Dit is een **live lesson**:

- Click = *hele document weg*
- Student snapt meteen waarom dit verouderd is

Mini-opdracht

Toon aan dat **moderne DOM output beter is**.

Maak een vergelijkingskaart met 2 knoppen:

Knop	Actie
"Write old way"	<code>document.write("🤖 Oeps")</code>
"Write modern way"	update een <div> met mooie Bootstrap alert

ID's

ow_old
ow_new
ow_out

Doel

Output 1 (slecht):

🤖 pagina stuk

Output 2 (goed):

Bootstrap success alert:

Moderne DOM output is beter!

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-warning text-dark">Mini-opdracht: oud vs
nieuw</div>
  <div class="card-body">

    <button id="ow_old" class="btn btn-outline-danger w-100 mb-2">Old way
X</button>
    <button id="ow_new" class="btn btn-outline-success w-100 mb-2">Modern
</button>

    <div id="ow_out" class="alert alert-secondary mb-0">
      Wacht op een keuze...
    </div>

  </div>
</div>
```

JS

```
function oudOutput() {
  alert("Nu breekt de pagina 🤖");
  document.write("🤖 Oeps, document.write overschreef alles!");
}

function nieuwOutput() {
```

```

const box = document.getElementById("ow_out");
box.className = "alert alert-success mb-0";
box.textContent = " Moderne DOM output is beter!";
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("ow_old")?.addEventListener("click", oudOutput);
  document.getElementById("ow_new")?.addEventListener("click", nieuwOutput);
});

```

Samenvatting

Methode	Gebruik	Status
document.write()	pagina direct overschrijven	✗ legacy
textContent, innerHTML	tekst/HTML tonen	modern
DOM methods	UI bouwen	aanbevolen

Document.write = no-go in moderne webapps.

Hoofdstuk 11: innerHTML VS textContent VS innerText

Doel van dit hoofdstuk

Na deze module kan je:

- het verschil uitleggen tussen innerHTML, textContent en innerText
- veilig tekst tonen in de UI
- HTML injecteren wanneer het WEL mag
- beschermen tegen gebruikersinput die HTML bevat
- een realistische interface bouwen met visuele feedback

Dit hoofdstuk is waar studenten vaak denken “Ah ok... nu snap ik het.”

Theorie

textContent

Toont ruwe tekst. Negeert HTML. Veilig.

```
element.textContent = "<b>Hallo</b>";
```

Resultaat op pagina:

```
<b>Hallo</b>
```

Gebruik wanneer: Je content is tekst Je wilt HTML niet uitvoeren Security is belangrijk

innerHTML

Voert HTML uit. Kan styling en tags toevoegen.

```
element.innerHTML = "<b>Hallo</b>";
```

Resultaat op pagina:

Hallo

Gebruik wanneer: Je wilt HTML inzetten  Alleen met veilige content

innerText

Toont tekst zoals zichtbaar in de browser. Houdt rekening met:

- CSS
- zichtbaarheid
- line breaks

Wordt minder gebruikt dan de andere twee.

Kort overzicht

Methode	Voert HTML uit?	Beste gebruik
textContent	Nee	Veilig tekst tonen
innerHTML	Ja	Dynamische HTML, icons, kleur
innerText	Nee, maar format volgt UI	Tekst zoals gebruiker ziet

Veiligheid tip

Gebruik NOOIT direct innerHTML met onbekende user-input.

Dit is hoe hackers scripts injecteren.

We tonen dat zo meteen.

Demo: drie output methoden

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">
    Hoofdstuk 11: innerHTML vs textContent vs innerText
  </div>

  <div class="card-body">
    <input id="ht_input" class="form-control mb-2" placeholder="Typ iets met
<b>HTML</b>">

    <button id="ht_btn_show" class="btn btn-info w-100 mb-3">
      Toon resultaten
    </button>

    <div class="alert alert-secondary mb-2">textContent:</div>
    <div id="ht_textContent" class="border p-2"></div>

    <div class="alert alert-secondary mt-3 mb-2">innerText:</div>
    <div id="ht_innerText" class="border p-2"></div>

    <div class="alert alert-secondary mt-3 mb-2">innerHTML:</div>
    <div id="ht_innerHTML" class="border p-2"></div>
```

```
</div>
</div>
```

main.js

```
// -----
// Hoofdstuk 11: innerHTML vs textContent vs innerText
// -----

function toonOutputTypes() {
  const input = document.getElementById("ht_input").value;

  document.getElementById("ht_textContent").textContent = input;
  document.getElementById("ht_innerText").innerText = input;
  document.getElementById("ht_innerHTML").innerHTML = input;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("ht_btn_show")
    ?.addEventListener("click", toonOutputTypes);
});
```

Test met:

```
<b>Hallo</b>
```

Je zal meteen zien:

- textContent toont letterlijk Hallo
- innerHTML maakt het vet
- innerText werkt bijna hetzelfde maar behoudt browserstijl logica

Mini-opdracht

Maak een inputveld waarin studenten HTML proberen te typen, maar jij toont het altijd veilig.

UI eisen

- Input veld
- Knop: "Veilig tonen"
- Output card
- Geen HTML mag uitgevoerd worden

ID's

safe_input
safe_btn
safe_output

Hint

```
safe_output.textContent = userInput;
```

Oplossing mini-opdracht

HTML

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-dark text-white">Mini-opdracht: Veilige  
output</div>  
  <div class="card-body">  
  
    <input id="safe_input" class="form-control mb-2" placeholder="Typ HTML  
code">  
  
    <button id="safe_btn" class="btn btn-dark w-100 mb-2">Veilig  
tonen</button>  
  
    <div id="safe_output" class="alert alert-secondary mb-0">  
      Wacht op input...  
    </div>  
  
  </div>  
</div>
```

JS

```
function toonVeilig() {  
  const val = document.getElementById("safe_input").value;  
  const out = document.getElementById("safe_output");  
  
  out.className = "alert alert-success mb-0";  
  out.textContent = val;  
}  
  
document.addEventListener("DOMContentLoaded", () => {  
  document.getElementById("safe_btn")  
    ?.addEventListener("click", toonVeilig);  
});
```


Samenvatting

Methode	Gebruik
textContent	Veilig tekst tonen, voorkom XSS
innerHTML	HTML uitvoeren, styling, icons
innerText	Tekst tonen zoals zichtbaar op pagina

Belangrijkste les:

Gebruik `textContent` voor user input. Gebruik `innerHTML` alleen voor **jouw eigen veilige strings**.

Hoofdstuk 12: DOM Selectors

Leerdoelen

Na dit hoofdstuk kan je:

- elementen selecteren met verschillende DOM-methodes
- het verschil begrijpen tussen **single** en **multiple** selectors
- `getElementById`, `getElementsByClassName`, `querySelector`, `querySelectorAll` gebruiken
- elementen dynamisch manipuleren in UI
- lijstitems filteren op basis van input

Theorie: DOM Selectors

Wanneer je de DOM wil gebruiken, moet je **iets kunnen aanwijzen**. Dit zijn je tools:

Methode	Teruggeven	Gebruik
<code>getElementById()</code>	1 element	snel & meest gebruikt
<code>getElementsByClassName()</code>	HTMLCollection	meerdere elementen met een class
<code>querySelector()</code>	1 element	CSS-selector stijl
<code>querySelectorAll()</code>	NodeList	meerdere elementen met CSS-selectors

Voorbeelden

getElementById

```
const title = document.getElementById("titel");
```

getElementsByClassName

```
const items = document.getElementsByClassName("item");
```

geeft een HTMLCollection → lijkt op array maar is geen echte array

querySelector

```
const btn = document.querySelector(".btn-primary");
```

querySelectorAll

```
const links = document.querySelectorAll("a.link");
```

geeft een NodeList → kan je **forEach** op gebruiken

Denkbeeld

Stel de DOM voor als een speelterrein. Deze functions zijn **zoeklichten**:

- `getElementById` → “zoek dat ene ding daar”
- `getElementsByClassName` → “pak alle elementen in deze rij”
- `querySelector` → “haal het eerste voorbeeld van dit patroon”
- `querySelectorAll` → “haal alles dat past op dit patroon”

Demo: lijst filteren met `querySelectorAll`

We maken:

- inputveld
- lijst van namen
- live filter op basis van wat je typt

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">
    Hoofdstuk 12: DOM Selectors
  </div>
  <div class="card-body">

    <label class="form-label">Zoek een naam:</label>
    <input id="sel_input" class="form-control mb-3" placeholder="bv. Tom">

    <ul id="sel_list" class="list-group">
      <li class="list-group-item naam-item">Tom</li>
      <li class="list-group-item naam-item">Sara</li>
      <li class="list-group-item naam-item">Liam</li>
      <li class="list-group-item naam-item">Emma</li>
      <li class="list-group-item naam-item">Noah</li>
    </ul>

  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 12: DOM Selectors
// -----

function filterNamen() {
  const value = document.getElementById("sel_input").value.toLowerCase();
  const items = document.querySelectorAll(".naam-item"); // NodeList

  items.forEach(item => {
    const text = item.textContent.toLowerCase();
    item.style.display = text.includes(value) ? "block" : "none";
  });
}
```

```
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("sel_input")
    ?.addEventListener("keyup", filterNamen);
});
```

Live filtering Simpele UX Klassieke querySelectorAll praktijk

Mini-opdracht

Maak een **kleurenswitcher**:

UI

- Drie knoppen: **rood, blauw, groen**
- Alle `.kleur-box` elementen veranderen naar die kleur

IDs & Classes

```
ks_red
ks_blue
ks_green
kleur-box
```

Hints

```
const boxes = document.querySelectorAll(".kleur-box");
```

Loop door de boxes en verander de achtergrondkleur.

Oplossing mini-opdracht

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht:
  Kleurenswitcher</div>
  <div class="card-body">

    <button id="ks_red" class="btn btn-danger w-100 mb-2">Rood</button>
    <button id="ks_blue" class="btn btn-primary w-100 mb-2">Blauw</button>
    <button id="ks_green" class="btn btn-success w-100 mb-3">Groen</button>

    <div class="d-flex gap-2">
      <div class="kleur-box p-3 border flex-fill text-center">Box 1</div>
      <div class="kleur-box p-3 border flex-fill text-center">Box 2</div>
      <div class="kleur-box p-3 border flex-fill text-center">Box 3</div>
    </div>

  </div>
</div>
```

JS

```
function setColor(color) {  
  const boxes = document.querySelectorAll(".kleur-box");  
  boxes.forEach(box => box.style.background = color);  
}  
  
document.addEventListener("DOMContentLoaded", () => {  
  document.getElementById("ks_red")?.addEventListener("click", () =>  
    setColor("red"));  
  document.getElementById("ks_blue")?.addEventListener("click", () =>  
    setColor("blue"));  
  document.getElementById("ks_green")?.addEventListener("click", () =>  
    setColor("green"));  
});
```

Samenvatting

Selector	Gebruik
getElementById	1 element, snel
getElementsByClassName	collectie, geen echte array
querySelector	1 element, CSS selectors
querySelectorAll	meerdere, NodeList met forEach

Jij kan nu de DOM **gericht doorzoeken en manipuleren**. Dat is web-toverkunst.

Hoofdstuk 13: Andere DOM-element properties & acties

Leerdoelen

Na deze module kan je:

- `.focus()` gebruiken om een veld actief te zetten
- `.blur()` gebruiken om focus weg te halen
- `.children` en `.childNodes` herkennen
- `.children.length` gebruiken om het aantal child-elementen te tellen
- interactieve UI-feedback bouwen

Theorie

`.focus()`

Zet het cursor-focus in een element.

```
document.getElementById("veld").focus();
```

`.blur()`

Haalt de focus weg.

```
document.getElementById("veld").blur();
```

Gebruiksmomenten:

Waarvoor?
Automatisch cursor zetten in input
Focus verplaatsen na actie
Velden laten "oplichten" bij focus

`.children`

Alle **element**-kinderen van een node.

`container.children`

`.childNodes`

Alles: whitespace, tekst, comments, elementen.

`container.childNodes`

`.children.length`

Aantal echte HTML-kinderen.

Vergelijking

Property	Wat zit erin?	Gebruik
<code>.children</code>	Alleen elementen	UI manipulatie
<code>.childNodes</code>	Tekst, spaties, comments, elementen	Debugging, zeldzaam nodig

Metafoor

Stel je DOM voor als een huis.

- `.children` = echte kamers
- `.childNodes` = kamers + **stof in de hoek + post-it op deur + lucht**

We willen meestal kamers, niet stof.

Demo: focus highlight + child counter

We maken:

- 2 inputvelden
- knop om focus te zetten
- info-vak dat telt hoeveel children een container heeft

`index.html`

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-warning text-dark">
    Hoofdstuk 13: DOM Focus & Children
  </div>
  <div class="card-body">

    <label class="form-label">Naam:</label>
    <input id="d13_name" class="form-control mb-2" placeholder="bv. Tom">

    <label class="form-label">Email:</label>
```

```

    <input id="d13_email" class="form-control mb-3" placeholder="bv.
tom@mail.be">

    <button id="d13_focusName" class="btn btn-warning w-100 mb-2">Focus op
Naam</button>
    <button id="d13_focusEmail" class="btn btn-dark w-100 mb-2">Focus op
Email</button>

    <div class="mt-3 alert alert-secondary">
        Totaal elementen in card-body: <span id="d13_count"></span>
    </div>

</div>
</div>

```

main.js

```

// -----
// Hoofdstuk 13: focus(), blur(), children
// -----

function updateChildCount() {
    const container = document.querySelector(".card-body");
    const count = container.children.length;
    document.getElementById("d13_count").textContent = count;
}

// Event binding
document.addEventListener("DOMContentLoaded", () => {
    updateChildCount();

    document.getElementById("d13_focusName")
        ?.addEventListener("click", () => {
            document.getElementById("d13_name").focus();
        });

    document.getElementById("d13_focusEmail")
        ?.addEventListener("click", () => {
            document.getElementById("d13_email").focus();
        });
});

```

Mini-opdracht

Maak een **focus training spelletje**:

Doel

Als de student op “Start oefening” klikt:

- input krijgt focus
- border wordt blauw bij focus
- border wordt grijs bij blur
- een teller toont hoeveel velden de container heeft

UI

- Input
- Knop: “Start oefening”
- Alert met child count

Verplichte ID's

ft_input
ft_btn
ft_count

Hints

```
input.addEventListener("focus", ...)  
input.addEventListener("blur", ...)
```

Border kleur tip:

```
input.style.border = "2px solid blue";
```

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-info text-white">Mini-opdracht: Focus Game</div>  
  <div class="card-body" id="ft_body">  
  
    <input id="ft_input" class="form-control mb-2" placeholder="Klik  
start...">  
    <button id="ft_btn" class="btn btn-info w-100 mb-2">Start  
oefening</button>  
  
    <div class="alert alert-secondary mb-0">  
      Kind elementen: <span id="ft_count">0</span>  
    </div>  
  
  </div>  
</div>
```

JS

```
function updateFTCount() {
  const body = document.getElementById("ft_body");
  document.getElementById("ft_count").textContent = body.children.length;
}

document.addEventListener("DOMContentLoaded", () => {
  const inp = document.getElementById("ft_input");

  updateFTCount();

  inp.addEventListener("focus", () => {
    inp.style.border = "2px solid blue";
  });

  inp.addEventListener("blur", () => {
    inp.style.border = "1px solid #ccc";
  });

  document.getElementById("ft_btn")?.addEventListener("click", () => {
    inp.focus();
  });
});
```

Samenvatting

Concept	Wat je nu weet
.focus()	Zet cursor in veld
.blur()	Haalt focus weg
.children	Alleen echte elementen
.childNodes	Alles (tekst, comments, whitespace)
.children.length	Tel elementen

Jij stuurt nu de DOM als een chauffeur die de weg kent.

Hoofdstuk 14: Klassen manipuleren (`classList`)

Leerdoelen

Na dit hoofdstuk kan je:

- CSS-klassen toevoegen, verwijderen en togglen via JS
- `.classList.add()`, `.remove()`, `.toggle()` gebruiken
- UI interactief dynamisch maken
- actieve knoppen en dark-mode gedrag bouwen

Theorie

`element.classList`

Geeft toegang tot de lijst met klassen van een element.

Belangrijkste methodes

Methode	Uitleg
<code>.add("naam")</code>	klasse toevoegen
<code>.remove("naam")</code>	klasse verwijderen
<code>.toggle("naam")</code>	toevoegen of verwijderen afhankelijk van status

Voorbeelden

```
box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("highlight");
```

Waarom belangrijk?

Interactieve UI Dark mode Actieve menu's Accordion gedrag Animaties triggeren

Alles modern aan UI komt van **klassen schakelen**.

Demo: Highlight toggle + knop states

We bouwen:

- 3 boxen
- 1 knop: "Highlight wisselen"
- Klik = highlight CSS aan/uit

`index.html`

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-success text-white">
    Hoofdstuk 14: Klassen manipuleren
  </div>
  <div class="card-body">

    <button id="cl_toggle" class="btn btn-success w-100 mb-3">
      Toggle Highlight
    </button>
```

```

    <div class="d-flex gap-2">
      <div class="cl-box p-3 border flex-fill text-center bg-light">Box
1</div>
      <div class="cl-box p-3 border flex-fill text-center bg-light">Box
2</div>
      <div class="cl-box p-3 border flex-fill text-center bg-light">Box
3</div>
    </div>
  </div>
</div>

```

CSS (voeg toe aan styles.scss onderaan)

```

.highlight {
  background: gold;
  font-weight: bold;
  transition: 0.2s ease;
}

```

main.js

```

// -----
// Hoofdstuk 14: classList
// -----

function toggleHighlight() {
  const boxes = document.querySelectorAll(".cl-box");
  boxes.forEach(box => box.classList.toggle("highlight"));
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("cl_toggle")
    ?.addEventListener("click", toggleHighlight);
});

```

Je hebt nu **elementen die reageren als echte UI widgets**

Mini-opdracht

Maak een **Dark / Light Theme toggle**.

UI

- 1 knop: "Dark mode"
- Wanneer actief:
 - body krijgt klasse dark-mode
 - knoptekst verandert naar "Light mode"
 - alert toont status

Verplichte ID's

dm2_btn

dm2_status

CSS toevoegen

```
.dark-mode {  
  background: #111 !important;  
  color: #fff !important;  
}
```

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-dark text-white">Mini-opdracht: Dark Mode  
Toggle</div>  
  <div class="card-body">  
  
    <button id="dm2_btn" class="btn btn-dark w-100 mb-2">Dark mode  
🌙</button>  
  
    <div id="dm2_status" class="alert alert-secondary mb-0">Light mode actief  
☀️</div>  
  
  </div>  
</div>
```

JS

```
function toggleDarkMode() {  
  const body = document.body;  
  const btn = document.getElementById("dm2_btn");  
  const status = document.getElementById("dm2_status");  
  
  const isDark = body.classList.toggle("dark-mode");  
  
  if (isDark) {  
    btn.textContent = "Light mode ☀️";  
    status.className = "alert alert-success mb-0";  
    status.textContent = "Dark mode actief 🌙";  
  } else {
```

```

    btn.textContent = "Dark mode 🌙";
    status.className = "alert alert-secondary mb-0";
    status.textContent = "Light mode actief 🌞";
  }
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("dm2_btn")
    ?.addEventListener("click", toggleDarkMode);
});

```

Klasse toggle Dynamische tekst UI state management Realtime styling

Samenvatting

Methode	Action
<code>.classList.add()</code>	zet CSS klasse
<code>.classList.remove()</code>	verwijdert CSS klasse
<code>.classList.toggle()</code>	schakel aan/uit

Jii kan nu **state, styling en interactie combineren**. Dit is hoe je webapps laat voelen als apps, niet documenten.

Hoofdstuk 15: Events & Formulieren

Leerdoelen

Na dit hoofdstuk kan je:

- `addEventListener` correct gebruiken
- click-, input- en submit-events verwerken
- formulieren uitlezen en valideren
- foutmeldingen tonen met Bootstrap alerts
- dynamische feedback geven tijdens typen
- HTML formuliergedrag begrijpen

Theorie: Wat is een Event?

Een **event** is iets dat gebeurt in de browser:

Gebeurtenis	Voorbeeld
Click	gebruiker klikt knop
Input	gebruiker typt in veld
Submit	formulier wordt verstuurd
Mouseover	muis beweegt over element
Focus / Blur	veld wordt actief/inactief

JS luistert naar die gebeurtenissen. Wanneer iets gebeurt, **voeren we een functie uit**.

.

Event Listener

```
element.addEventListener("eventnaam", functie);
```

Voorbeeld:

```
btn.addEventListener("click", sayHello);
```

Belangrijk

Fout	Waarom
onclick="..."	oud, rommelig, inline scripts
addEventListener	modern, proper, schaalbaar

event.preventDefault()

Blokkeert het **standaard gedrag** (form submit, link klik,...)

We gebruiken dit zo meteen bij forms.

Metafoor

Events zijn als **belletjes aan deuren**.

Je hangt een bel (listener) en wanneer iemand duwt → er gebeurt iets.

Demo: Live naam weergave + form submit

We bouwen:

- input: naam
- live preview tijdens typen
- submit knop
- validatie: verplicht veld
- success- en error alerts

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">
    Hoofdstuk 15: Events & Formulieren
  </div>
  <div class="card-body">

    <form id="ev_form">

      <label class="form-label">Naam:</label>
      <input id="ev_name" class="form-control mb-2" placeholder="bv. Tom">

      <div id="ev_preview" class="alert alert-secondary mb-2">
        Wacht op input...
      </div>

      <button type="submit" class="btn btn-primary w-100">
        Verzenden
      </button>
    </form>
  </div>
</div>
```

```

    </form>

    <div id="ev_status" class="alert alert-secondary mt-3 mb-0">
        Formulier niet verstuurd
    </div>

</div>
</div>

main.js
// -----
// Hoofdstuk 15: Events & Formulieren
// -----

function updatePreview() {
    const name = document.getElementById("ev_name").value;
    const preview = document.getElementById("ev_preview");

    if (name.trim() === "") {
        preview.className = "alert alert-secondary mb-2";
        preview.textContent = "Wacht op input...";
    } else {
        preview.className = "alert alert-info mb-2";
        preview.textContent = `Hallo ${name}! 🙌`;
    }
}

function handleSubmit(event) {
    event.preventDefault(); // stop page reload

    const name = document.getElementById("ev_name").value.trim();
    const status = document.getElementById("ev_status");

    if (!name) {
        status.className = "alert alert-danger mb-0 mt-3";
        status.textContent = "❌ Vul een naam in";
        return;
    }

    status.className = "alert alert-success mb-0 mt-3";
    status.textContent = `Formulier verstuurd voor ${name}`;
}

document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("ev_name")?.addEventListener("input",
updatePreview);
    document.getElementById("ev_form")?.addEventListener("submit",
handleSubmit);
});

```

Mini-opdracht

Maak een **email validatie formulier**.

UI eisen

- input: email
- knop: "Check email"
- live "✓ geldig" / "✗ ongeldig" tijdens typen
- submit toont succesmelding alleen bij geldig emailadres

IDs

em_input
em_form
em_preview
em_status

Validatieregels

- moet @ bevatten
- moet een . bevatten (na de @)
- geen spaties

Hint:

```
value.includes("@")
```

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-dark text-white">Mini-opdracht: Email  
  checker</div>  
  <div class="card-body">  
  
    <form id="em_form">  
  
      <input id="em_input" class="form-control mb-2" placeholder="email">  
      <div id="em_preview" class="alert alert-secondary mb-2">Typ een  
email...</div>  
  
      <button type="submit" class="btn btn-dark w-100">Check email</button>  
  
    </form>  
  
    <div id="em_status" class="alert alert-secondary mt-3 mb-0">  
      Geen actie uitgevoerd  
    </div>  
  
  </div>  
</div>
```

JS

```
function isValidEmail(email) {
  return email.includes("@") && email.includes(".") && !email.includes(" ");
}

function updateEmailPreview() {
  const value = document.getElementById("em_input").value;
  const preview = document.getElementById("em_preview");

  if (isValidEmail(value)) {
    preview.className = "alert alert-success mb-2";
    preview.textContent = " Geldig emailadres";
  } else {
    preview.className = "alert alert-warning mb-2";
    preview.textContent = "⚠ Ongeldig emailadres";
  }
}

function handleEmailSubmit(e) {
  e.preventDefault();

  const val = document.getElementById("em_input").value;
  const status = document.getElementById("em_status");

  if (!isValidEmail(val)) {
    status.className = "alert alert-danger mb-0 mt-3";
    status.textContent = "✖ Emailadres ongeldig";
    return;
  }

  status.className = "alert alert-success mb-0 mt-3";
  status.textContent = `Email geaccepteerd: ${val}`;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("em_input")?.addEventListener("input",
    updateEmailPreview);
  document.getElementById("em_form")?.addEventListener("submit",
    handleEmailSubmit);
});
```

Samenvatting

Concept	Wat je nu kan
addEventListener	moderne event binding
input event	real-time UI feedback
submit + preventDefault()	formulieren controleren
Bootstrap alerts	mooie feedback
Validatie	eerste stap naar echte forms

Jij hebt nu **de basis van every web app ever**.

Hoofdstuk 16: Event Types & `preventDefault()` (Dieper) + Mouse Events

Leerdoelen

Na dit hoofdstuk kan je:

- uitleggen wat `preventDefault()` precies blokkeert
- het verschil snappen tussen **default browser gedrag** en **custom JS gedrag**
- mouse events gebruiken (`click`, `mouseover`, `mouseleave`)
- buttons & links verschillend behandelen
- debuggen: wanneer event *bubbelt* en waarom
- UI-feedback geven op mouse events

Dit hoofdstuk geeft studenten “*aha, daarom doet m’n pagina dat!*”-energie.

Theorie

Default browser acties

Actie	Normaal gedrag
klik op link	pagina navigeert weg
submit form	pagina herlaadt + verstuurt data
drag/drop	browser probeert te verplaatsen
rechterklik	toont context-menu

Wat doet `event.preventDefault()`?

Je zegt tegen de browser: “Stop. Ik neem over.”

```
event.preventDefault();
```

Mouse events

Event	Moment
<code>click</code>	klik
<code>mouseover</code>	muis komt erover
<code>mouseleave</code>	muis gaat weg
<code>mousedown</code>	muisknop ingedrukt
<code>mouseup</code>	muisknop loslaat

Waarom essentieel?

Nodig bij SPA-gedrag, Nodig bij AI search bar input, Nodig bij forms zonder reload, Nodig bij custom UI's (dropdowns, modals, menus)

Dit is **één van de top 3 skills** in front-end.

Demo: Link + Box hover effect + Submit block

We bouwen:

- link die normaal navigeert
- JS blokkeert navigatie & geeft melding
- box highlight bij hover
- submit-button zonder reload

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-secondary text-white">
    Hoofdstuk 16: Event Types & preventDefault
  </div>
  <div class="card-body">

    <a id="ev_link" href="https://google.com" class="d-block mb-2">
      Link naar Google (wordt tegengehouden)
    </a>

    <div id="ev_box" class="p-4 text-center border mb-3 bg-light">
      Hover over mij ⓘ
    </div>

    <form id="ev_form2">
      <input id="ev_input2" class="form-control mb-2" placeholder="Naam">
      <button class="btn btn-secondary w-100">Verzend (JS blokkeert)</button>
    </form>

    <div id="ev_msg2" class="alert alert-secondary mt-3 mb-0">
      Wacht op interactie...
    </div>

  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 16: Events + preventDefault()
// -----

document.addEventListener("DOMContentLoaded", () => {

  // 1) Link blokkeren
  document.getElementById("ev_link")
    ?.addEventListener("click", (e) => {
      e.preventDefault();
      document.getElementById("ev_msg2").textContent =
        "ⓘ Navigatie geblokkeerd door JS";
    });

  // 2) Hover effects
  const box = document.getElementById("ev_box");
```

```

box.addEventListener("mouseover", () => {
  box.classList.add("bg-warning");
  box.textContent = "Hover actief!";
});

box.addEventListener("mouseleave", () => {
  box.classList.remove("bg-warning");
  box.textContent = "Hover over mij 🐱";
});

// 3) Form prevent + feedback
document.getElementById("ev_form2")
  ?.addEventListener("submit", (e) => {
    e.preventDefault();

    const name = document.getElementById("ev_input2").value.trim();
    const msg = document.getElementById("ev_msg2");

    if (!name) {
      msg.className = "alert alert-danger mt-3 mb-0";
      msg.textContent = "❌ Vul je naam in";
      return;
    }

    msg.className = "alert alert-success mt-3 mb-0";
    msg.textContent = `Form verzonden voor ${name} (zonder reload!)`;
  });
});

```

Mini-opdracht

Maak een “**Reaction Box**”

Wat moet het kunnen:

Event	Actie
mouseover	tekst = “Hallo!” & achtergrond groen
mouseleave	tekst = “Kom terug!” & achtergrond licht
click	tekst = “Je klikt!” & rand dik
dblclick	tekst = “Dubbelklik wow!” & rand rood

IDs

rb_box

rb_msg

Gebruiker voelt UI *leven*

Oplossing mini-opdracht

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">
    Mini-opdracht: Reaction Box
  </div>
  <div class="card-body">

    <div id="rb_box" class="p-4 text-center border bg-light mb-3">
      Hover / click mij
    </div>

    <div id="rb_msg" class="alert alert-secondary mb-0">
      Wacht op interactie...
    </div>
  </div>
</div>
```

JS

```
document.addEventListener("DOMContentLoaded", () => {
  const box = document.getElementById("rb_box");
  const msg = document.getElementById("rb_msg");

  box.addEventListener("mouseover", () => {
    box.style.background = "lightgreen";
    msg.textContent = "👋 Hallo!";
  });

  box.addEventListener("mouseleave", () => {
    box.style.background = "#f8f9fa";
    msg.textContent = "😞 Kom terug!";
  });

  box.addEventListener("click", () => {
    box.style.border = "3px solid black";
    msg.textContent = "👉 Je klikt!";
  });

  box.addEventListener("dblclick", () => {
    box.style.border = "3px solid red";
    msg.textContent = "🔪 Dubbelklik wow!";
  });
});
```

Samenvatting

Concept	Wat je nu kan
preventDefault	browser-gedrag blokkeren
mouseover / leave	hover-logica bouwen
click / dblclick	actie interacties
form events	UI zonder page refresh

Concept	Wat je nu kan
alert messaging	realtime feedback

Jij begint nu te denken als een **Frontend-engineer**.

Hoofdstuk 17: Mini-Project – Todo App (Events + Arrays + DOM)

Doel van dit hoofdstuk

Na dit project kan je:

- taken opslaan in een array
- taken toevoegen via form
- fouten tonen als input leeg is
- taken markeren als voltooid via `.toggle()`
- taken verwijderen
- een real-world component bouwen met Bootstrap

Uitleg

Een **todo-app** is perfect om event-logica te trainen.

We bouwen:

Feature	Beschrijving
Task input	gebruiker typt taak in
Toevoegen	knop of enter
Weergave	lijst van tasks
Markeren	klik = done toggle (CSS class)
Verwijderen	klik op delete-knop
UI feedback	alerts voor errors

UI Design

We gebruiken Bootstrap en een **list-group** layout, net als real SaaS apps.

CSS extra (voeg onderaan je `styles.scss` toe)

```
.todo-done {
  text-decoration: line-through;
  opacity: 0.6;
}
```

index.html

Voeg dit kaart-blok toe:

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-success text-white">
    Hoofdstuk 17: Todo App Mini-Project
  </div>
  <div class="card-body">

    <form id="todo_form2" class="mb-3">
      <input id="todo_input2" class="form-control mb-2" placeholder="Nieuwe
taak">
      <button class="btn btn-success w-100">Voeg toe</button>
    </form>

    <div id="todo_msg2" class="alert alert-secondary mb-3">
      Voeg een taak toe om te beginnen
    </div>

    <ul id="todo_list2" class="list-group"></ul>
  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 17: Todo app mini-project
// -----

const taken2 = [];

function renderTodos() {
  const list = document.getElementById("todo_list2");

  list.innerHTML = taken2
    .map((task, index) => `
    <li class="list-group-item d-flex justify-content-between align-items-
center ${task.done ? "todo-done" : ""}">
      <span class="todo-text" data-idx="${index}">
        ${task.text}
      </span>
      <button class="btn btn-sm btn-danger todo-del" data-
idx="${index}">X</button>
    </li>
  `)
    .join("");
}

function addTodo(e) {
  e.preventDefault();

  const inp = document.getElementById("todo_input2");
  const msg = document.getElementById("todo_msg2");
  const text = inp.value.trim();
```

```

    if (!text) {
      msg.className = "alert alert-danger mb-3";
      msg.textContent = "✗ Vul een taak in";
      return;
    }

    taken2.push({ text, done: false });

    msg.className = "alert alert-success mb-3";
    msg.textContent = `Taak toegevoegd: ${text}`;

    inp.value = "";
    renderTodos();
  }

  function handleTodoClick(e) {
    const idx = e.target.dataset.idx;

    // Toggle done
    if (e.target.classList.contains("todo-text")) {
      taken2[idx].done = !taken2[idx].done;
      renderTodos();
      return;
    }

    // Delete
    if (e.target.classList.contains("todo-del")) {
      taken2.splice(idx, 1);
      renderTodos();
    }
  }

  document.addEventListener("DOMContentLoaded", () => {
    document.getElementById("todo_form2")?.addEventListener("submit", addTodo);
    document.getElementById("todo_list2")?.addEventListener("click",
    handleTodoClick);
  });

```

Demo features die studenten leren

- formulier + preventDefault
- input uitlezen
- array manipuleren
- re-renderen via map() + .join()
- data-attributes (data-idx)
- class toggling
- delete met splice
- Bootstrap alerts

Dit is **levende applicatie-logica**.

Mini-Opdracht

Breid de todo-app uit met:

Feature	Beschrijving
! Filter	drie knoppen: Alle , Open , Voltooid
📊 Counter	bovenaan: Aantal open taken: X
🧹 Clear done	knop om alle voltooide taken te verwijderen

Maak UI alvast slim:

- Knoppen gebruik btn-outline-*
- Active knop krijgt btn-*

ID's

```
todo_filter_all  
todo_filter_open  
todo_filter_done  
todo_clear_done  
todo_count_open
```

Samenvatting

Je hebt nu gebouwd:

- echte UI
- state management (array)
- events & listeners
- toggles en delete
- feedback messages

Dit is een **mini-SPA zonder framework**. Je studenten voelen nu controle.

Hoofdstuk 18: Objects & Date() in JavaScript

Leerdoelen

Na dit hoofdstuk kan je:

- een object maken in JavaScript
- properties uitlezen en aanpassen
- het Date object gebruiken
- datumonderdelen uitlezen (jaar, maand, dag)
- een **leeftijd-calculator UI** bouwen
- datuminput verwerken via formulier

Theorie: Wat is een object?

Een object is een verzameling van eigenschappen (**key** → **value**).

Voorbeeld

```
const persoon = {  
  naam: "Tom",  
  leeftijd: 30,  
  stad: "Torhout"  
};
```

Waarden uitlezen:

```
console.log(persoon.naam);  
console.log(persoon["stad"]);
```

Nieuw veld toevoegen:

```
persoon.email = "test@mail.com";
```

Theorie: Date() object

Een Date object vertegenwoordigt datum en tijd.

```
const nu = new Date();
```

Belangrijkste methodes

Methode	Resultaat
.getFullYear()	huidig jaar
.getMonth()	maand (0-11)
.getDate()	dag (1-31)
.getDay()	weekdag (0-6)
.getHours()	uren

Voorbeeld huidig jaar

```
const jaar = new Date().getFullYear();
```

Use case: Leeftijd berekenen

Gebruiker geeft geboortjaar in → wij tonen leeftijd

Waarom? Veel toegepast: webshops, profielen, leeftijd-check systemen, etc.

Demo: Leeftijd-calculator

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">
    Hoofdstuk 18: Object & Date - Leeftijd Calculator
  </div>
  <div class="card-body">

    <form id="age_form">
      <label class="form-label">Geboortjaar:</label>
      <input id="age_input" type="number" class="form-control mb-2"
placeholder="bv. 1990">

      <button class="btn btn-info w-100 mb-2">Bereken leeftijd</button>
    </form>

    <div id="age_output" class="alert alert-secondary mb-0">
      Wacht op input...
    </div>

  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 18: Objects + Date()
// -----

function berekenLeeftijd(jaar) {
  const huidigJaar = new Date().getFullYear();
  return huidigJaar - jaar;
}

function toonLeeftijd(e) {
  e.preventDefault();

  const input = document.getElementById("age_input").value.trim();
  const out = document.getElementById("age_output");

  if (!input || isNaN(input)) {
    out.className = "alert alert-danger mb-0";
    out.textContent = "✗ Geef een geldig jaar in";
    return;
  }

  const leeftijd = berekenLeeftijd(Number(input));
```

```

const gebruiker = {
  geboorteJaar: Number(input),
  leeftijd: leeftijd
};

out.className = "alert alert-success mb-0";
out.textContent = `Je bent ongeveer ${gebruiker.leeftijd} jaar oud
(geboren in ${gebruiker.geboorteJaar})`;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("age_form")?.addEventListener("submit",
toonLeeftijd);
});

```

Mini-opdracht

Maak een **geboortedatum calculator**:

Student geeft in:

- dag
- maand
- jaar

Jij toont:

 Jij bent X jaar oud en geboren op een WEEKDAG

Voorbeeld:

 22 jaar – geboren op een woensdag

Verplichte ID's

dob_day
dob_month
dob_year
dob_btn
dob_out

Hint

```

const datum = new Date(jaar, maand - 1, dag);
const weekday = datum.getDay();

```

Zorg voor een array:

```

const dagen =
["Zondag", "Maandag", "Dinsdag", "Woensdag", "Donderdag", "Vrijdag", "Zaterdag"];

```

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Mini-opdracht:
  Geboortedag</div>
  <div class="card-body">

    <div class="row g-2 mb-2">
      <div class="col"><input id="dob_day" type="number" class="form-control"
placeholder="Dag"></div>
      <div class="col"><input id="dob_month" type="number" class="form-
control" placeholder="Maand"></div>
      <div class="col"><input id="dob_year" type="number" class="form-
control" placeholder="Jaar"></div>
    </div>

    <button id="dob_btn" class="btn btn-dark w-100 mb-2">Bereken</button>

    <div id="dob_out" class="alert alert-secondary mb-0">Wacht op
input...</div>

  </div>
</div>
```

JS

```
function berekenLeeftijdJaar(datum) {
  const nu = new Date();
  let leeftijd = nu.getFullYear() - datum.getFullYear();

  const isVoorVerjaardag =
    nu.getMonth() < datum.getMonth() ||
    (nu.getMonth() === datum.getMonth() && nu.getDate() < datum.getDate());

  if (isVoorVerjaardag) leeftijd--;

  return leeftijd;
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("dob_btn")?.addEventListener("click", () => {
    const d = Number(document.getElementById("dob_day").value);
    const m = Number(document.getElementById("dob_month").value);
    const y = Number(document.getElementById("dob_year").value);
    const out = document.getElementById("dob_out");

    if (!d || !m || !y) {
      out.className = "alert alert-danger mb-0";
      out.textContent = "✗ Vul dag, maand en jaar in";
      return;
    }

    const datum = new Date(y, m - 1, d);
```

```

    const dagen =
["Zondag", "Maandag", "Dinsdag", "Woensdag", "Donderdag", "Vrijdag", "Zaterdag"];
    const weekday = dagen[datum.getDay()];
    const age = berekenLeeftijdJaar(datum);

    out.className = "alert alert-success mb-0";
    out.textContent = `🎂 Je bent ongeveer ${age} jaar oud en geboren op een
    ${weekday}`;
  });
});

```

Samenvatting

Concept	Wat je leerde
Objecten	data opslaan met properties
Date()	werken met tijd & datum
Leeftijd berekenen	jaardelta + verjaardag check
User input verwerken	getal + validatie
UI feedback	Bootstrap alerts

Je hebt nu **objecten + tijd** onder de knie. Volgende stap = **Object constructors en prototypes (light)**.

Hoofdstuk 19: Object Constructors & Inheritance (Intro)

Leerdoelen

Na dit hoofdstuk kan je:

- begrijpen wat een **constructor function** is
- objecten maken via `new`
- zien hoe `this` werkt in object-context
- begrijpen wat **inheritance / prototype chain** betekent op beginnersniveau
- een mini-“user factory” bouwen
- UI feedback tonen met objectdata

Dit is **voorbereiding op Classes bij ES6**, maar nog zonder de class-syntax.

Theorie

Wat is een Constructor Function?

Een **constructor** is een functie die nieuwe objecten maakt.

Denk aan:

een koekjesvorm die koekjes maakt elke koek is uniek, maar de vorm is hetzelfde

Syntax

```
function User(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

Object maken:

```
const tom = new User("Tom", 30);
```

Wat doet new?

new zorgt ervoor dat:

Stap	Wat gebeurt
1	er ontstaat een nieuw leeg object { }
2	this verwijst naar dat object
3	properties worden toegevoegd
4	object wordt teruggegeven

Waarom belangrijk?

Omdat dit de basis is van:

- game characters
- product objects in webshops
- user sessions
- models in frameworks (Laravel Eloquent, Vue, React classes...)

Demo: User maken en tonen

We bouwen:

- inputs: naam + leeftijd
- knop maakt een User object via constructor
- lijst toont aangemaakte users

```

index.html
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">
    Hoofdstuk 19: Object Constructors
  </div>
  <div class="card-body">

    <input id="uc_name" class="form-control mb-2" placeholder="Naam">
    <input id="uc_age" type="number" class="form-control mb-2"
placeholder="Leeftijd">

    <button id="uc_btn" class="btn btn-primary w-100 mb-3">Maak
gebruiker</button>

    <ul id="uc_list" class="list-group"></ul>

  </div>
</div>

```

```

main.js
// -----
// Hoofdstuk 19: Constructor functions + object instances
// -----

const users = [];

function User(name, age) {
  this.name = name;
  this.age = age;
  this.info = function() {
    return `${this.name} (${this.age} jaar)`;
  };
}

function addUser() {
  const name = document.getElementById("uc_name").value.trim();
  const age = document.getElementById("uc_age").value.trim();
  const list = document.getElementById("uc_list");

  if (!name || !age) return;

  const user = new User(name, Number(age));
  users.push(user);

  list.innerHTML = users
    .map(u => `<li class="list-group-item">${u.info()}</li>`)
    .join("");
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("uc_btn")
    ?.addEventListener("click", addUser);
});

```

Je maakt **objecten uit een factory** Data komt uit UI Je toont het dynamisch Je voelt hoe this werkt

Mini-opdracht

Maak een Product constructor:

Eigenschap	Input
naam	product_name
prijs	product_price

Extra

Method `.label()` geeft terug:

🍕 Pizza – €12.50

UI requirements

- inputvelden
- knop
- ul-list

IDs

prod_name
prod_price
prod_btn
prod_list

Oplossing mini-opdracht

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">
    Mini-opdracht: Product Factory
  </div>
  <div class="card-body">

    <input id="prod_name" class="form-control mb-2"
placeholder="Productnaam">
    <input id="prod_price" type="number" class="form-control mb-2"
placeholder="Prijs">

    <button id="prod_btn" class="btn btn-dark w-100 mb-3">Voeg product
toe</button>

    <ul id="prod_list" class="list-group"></ul>
  </div>
</div>
```


JS

```
function Product(name, price) {
  this.name = name;
  this.price = price;
  this.label = function() {
    return `${this.name} - €${this.price.toFixed(2)}`;
  };
}

const products = [];

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("prod_btn").addEventListener("click", () => {
    const name = document.getElementById("prod_name").value.trim();
    const price = Number(document.getElementById("prod_price").value);

    if (!name || !price) return;

    const item = new Product(name, price);
    products.push(item);

    document.getElementById("prod_list").innerHTML = products
      .map(p => `<li class="list-group-item">${p.label()}</li>`)
      .join("");
  });
});
```

Inheritance Light

Simpel voorbeeld:

```
function Student(name, level) {
  User.call(this, name, 0);
  this.level = level;
}
```

Dat is de **oude** manier. We gebruiken dit later voor ES6 class extends.

Samenvatting

Concept	Jij kan nu
Constructor	functies als object-fabriek maken
new	nieuwe objecten genereren
this	verwijst naar het object dat je maakt
Object-methode	this.method = function()
UI + constructors	user-form naar objectlijst

Dit is de **brug naar classes**.

Hoofdstuk 20: ES6 Classes + extends + super

Leerdoelen

Na dit hoofdstuk kan je:

- een class definiëren
- constructor gebruiken
- objecten maken met new
- methods schrijven in een class
- een class **erfbaar** maken met extends
- super() gebruiken om de parent constructor te callen
- een mini user + admin systeem bouwen

Dit is de vorm die lijkt op OOP in PHP, Laravel, C#, Java.

Theorie: Wat is een Class?

Een **class** is een blauwdruk voor objecten. Elke instantie heeft eigen data, maar dezelfde structuur.

Syntax

```
class User {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  info() {  
    return `${this.name} (${this.age} jaar)`;  
  }  
}
```

Object maken

```
const tom = new User("Tom", 30);
```

Waarom classes gebruiken?

duidelijker dan constructor functions makkelijker uitbreiden basis van moderne frameworks lijkt op PHP/Laravel OOP (super relevant)

Inheritance met extends

```
class Admin extends User {
  constructor(name, age, role) {
    super(name, age); // oproep parent constructor
    this.role = role;
  }

  info() {
    return `[ADMIN] ${super.info()} - rol: ${this.role}`;
  }
}
```

Demo: User + Admin UI

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-secondary text-white">
    Hoofdstuk 20: ES6 Classes Demo
  </div>
  <div class="card-body">

    <input id="cls_name" class="form-control mb-2" placeholder="Naam">
    <input id="cls_age" type="number" class="form-control mb-2"
placeholder="Leeftijd">

    <select id="cls_role" class="form-select mb-3">
      <option value="user">User</option>
      <option value="admin">Admin</option>
    </select>

    <button id="cls_btn" class="btn btn-secondary w-100 mb-3">Maak
gebruiker</button>

    <ul id="cls_list" class="list-group"></ul>

  </div>
</div>
```

```

main.js
// -----
// Hoofdstuk 20: ES6 Classes + extends + super
// -----

class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  info() {
    return `${this.name} (${this.age} jaar)`;
  }
}

class Admin extends User {
  constructor(name, age) {
    super(name, age);
    this.role = "Admin";
  }

  info() {
    return `[ADMIN] ${super.info()}`;
  }
}

const clsUsers = [];

function addClassUser() {
  const name = document.getElementById("cls_name").value.trim();
  const age = Number(document.getElementById("cls_age").value);
  const role = document.getElementById("cls_role").value;
  const list = document.getElementById("cls_list");

  if (!name || !age) return;

  const user = role === "admin"
    ? new Admin(name, age)
    : new User(name, age);

  clsUsers.push(user);

  list.innerHTML = clsUsers
    .map(u => `<li class="list-group-item">${u.info()}</li>`)
    .join("");
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("cls_btn")
    ?.addEventListener("click", addClassUser);
});

```

moderne OOP in JS inheritance UI gekoppeld aan object instances

Mini-opdracht

Maak een simpele **dieren-class**:

Base class: `Animal`

Eigenschappen:

- name
- soort (bv "hond")
- method `speak()` retournt
🐾 {name} is een {soort}

Child class: `Dog` extends `Animal`

- override `speak()` met
🐕 {name} blaft!

UI

- input naam
- select soort (hond of kat)
- knop
- list

✦ Bonus: voeg emoji per dier

IDs

pet_name
pet_type
pet_btn
pet_list

Samenvatting

Concept	Jij kan nu
class	moderne object blauwdruk maken
constructor	eigenschappen aanvullen
new	object instanties bouwen
extends	child-class maken
super()	parent constructor en methods callen
UI verbinden	class data in list tonen

Jij schrijft nu JavaScript **zoals moderne frameworks verwachten**. React, Vue, Laravel Mix, Node? Deze skill komt steeds terug.

Hoofdstuk 21: Mini Project – Product Catalog (Webshop Logica)

Leerdoelen

Na dit hoofdstuk kan je:

- producten modelleren met een Product class
- een productlijst dynamisch in de DOM renderen
- een “Add to cart” knop bouwen
- een cart-counter badge tonen
- alerts tonen bij toevoegen
- state bewaren in geheugen

We bouwen een **realistische mini-webshop basis**.

Concept

Features

Feature	Uitleg
Product class	naam, prijs, emoji
Product lijst	tonen als cards
Buy button	klikt → in cart
Cart count badge	telt mee
Bootstrap UI	nette e-commerce look

We hebben geen database. State blijft in JS zolang de pagina open is.

Class Model

```
class Product {
  constructor(name, price, emoji) {
    this.name = name;
    this.price = price;
    this.emoji = emoji;
  }

  label() {
    return `${this.emoji} ${this.name} – €${this.price.toFixed(2)}`;
  }
}
```

HTML UI

Plaats dit onderaan je bestaande pagina:

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-success text-white d-flex justify-content-between align-items-center">
    Webshop Producten
    <span class="badge bg-warning text-dark" id="cart_count">0</span>
  </div>
  <div class="card-body" id="prod_container"></div>
```

```

</div>

<div id="cart_msg" class="alert alert-secondary mt-2 mb-0">
  Voeg een product toe aan je winkelmandje
</div>

```

Styles (optioneel animatie)

In styles.scss onderaan:

```

.cart-added {
  animation: pulse 0.3s ease;
}

@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.15); }
  100% { transform: scale(1); }
}

```

JavaScript

```

main.js
// -----
// Hoofdstuk 21: Mini Webshop Project
// -----

class Product {
  constructor(name, price, emoji) {
    this.name = name;
    this.price = price;
    this.emoji = emoji;
  }

  label() {
    return `${this.emoji} ${this.name} - €${this.price.toFixed(2)}`;
  }
}

const products3 = [
  new Product("Pizza", 12.5, "🍕"),
  new Product("Hamburger", 9.9, "🍔"),
  new Product("Sushi", 15.0, "🍣"),
  new Product("Frietjes", 4.5, "🍟")
];

let cartCount = 0;

function renderProducts() {
  const cont = document.getElementById("prod_container");

  cont.innerHTML = products3
    .map((p, i) => `
    <div class="card mb-2">

```

```

        <div class="card-body d-flex justify-content-between align-items-
center">
            <strong>${p.label()}</strong>
            <button class="btn btn-sm btn-success prod_buy" data-idx="${i}">
                Koop
            </button>
        </div>
    </div>
    `)
    .join("");
}

function addToCart(idx) {
    cartCount++;
    const badge = document.getElementById("cart_count");
    const msg = document.getElementById("cart_msg");

    badge.textContent = cartCount;
    badge.classList.add("cart-added");
    setTimeout(() => badge.classList.remove("cart-added"), 300);

    msg.className = "alert alert-success mt-2 mb-0";
    msg.textContent = ` ${products3[idx].name} toegevoegd aan winkelmandje`;
}

document.addEventListener("DOMContentLoaded", () => {
    renderProducts();

    document.getElementById("prod_container")
        ?.addEventListener("click", (e) => {
            if (e.target.classList.contains("prod_buy")) {
                addToCart(e.target.dataset.idx);
            }
        });
});
});

```

Feels like a real shop OOP + DOM + rendering loops Event delegation Cart indicator

Test checklist

Test	Resultaat
Klik "Koop"	badge ↑
Bericht zichtbaar	
Emoji & prijs netjes	
Multi-items werken	
Geen reloads	

Mini-opdracht (uitbreiding)

Voeg 2 functies toe:

Feature	Opdracht
Som tonen	toon totaalprijs
Leeg mandje	knop “Leeg mandje”

IDs toevoegen

cart_total
cart_clear

Hint

```
let cartItems = [];  
cartItems.push(products[idx]);
```

Totaal berekenen

```
const totaal = cartItems.reduce((s,p) => s + p.price, 0);
```

Laat studenten het bouwen. Hierna krijg je *jouw* begeleide oplossing.

Samenvatting

Je kan nu	Waarom belangrijk
Classes gebruiken	echte data-modellen maken
Producten renderen	basis voor catalog/repeater
Add to cart	e-commerce flow
State bijhouden	frontend architectuur
Bootstrap cards	real-world UI

Dit is **80% van een webshop's frontend** maar mini.

Hoofdstuk 22: Browser Object Model (BOM)

Leerdoelen

Na dit hoofdstuk kan je:

- uitleggen wat de BOM is
- window, screen, navigator herkennen
- URL info uitlezen
- user agent uitlezen
- browser redirect uitvoeren
- vensterfuncties begrijpen (zonder misbruik)
- kleine UI maken die BOM info toont

Theorie

Wat is de BOM?

DOM	BOM
Pagina elementen	Browser omgeving
document	window, screen, navigator, location
HTML structuur	Apparaat, browser, sessie

Het **BOM** geeft info en controle over:

Object	Purpose
window	globaal browserobject
screen	info over schermgrootte
navigator	browser & OS info
location	URL info + redirect

window

Alles in JS draait in window.

```
console.log(window.innerWidth);
```

Of korter (want default):

```
console.log(innerWidth);
```

screen

```
console.log(screen.width);  
console.log(screen.height);
```

navigator

```
console.log(navigator.userAgent);  
console.log(navigator.language);  
console.log(navigator.onLine);
```

location

```
console.log(location.href);
```

Redirect:

```
// location.href = "https://google.com";
```

Voor demo laten we redirect uit commentaar, maar niet automatisch uitvoeren.

Veiligheid: popups & redirects

✗ Geen dark patterns ✗ Geen auto-popups / -redirects

We tonen alleen **eerlijke UX voorbeelden**.

Demo: BOM Info Display

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">
    Hoofdstuk 22: Browser Object Model
  </div>
  <div class="card-body">

    <button id="bom_btn" class="btn btn-dark w-100 mb-3">Toon browser
    info</button>

    <ul id="bom_list" class="list-group mb-3"></ul>

    <button id="bom_go" class="btn btn-warning w-100">Ga naar Google</button>

  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 22: BOM Demo
// -----

function showBOM() {
  const data = [
    `🖥️ Window breedte: ${window.innerWidth}`,
    `🖥️ Screen resolutie: ${screen.width} x ${screen.height}`,
    `🌐 User Agent: ${navigator.userAgent}`,
    `💬 Browser taal: ${navigator.language}`,
    `🌐 Online: ${navigator.onLine ? "Ja" : "Nee"}`,
    `🔗 URL: ${location.href}`
  ];

  document.getElementById("bom_list").innerHTML =
    data.map(item => `<li class="list-group-item">${item}</li>`).join("");
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("bom_btn")
    ?.addEventListener("click", showBOM);
});
```

```
document.getElementById("bom_go")
  ?.addEventListener("click", () => {
    location.href = "https://google.com";
  });
});
```

UI geeft browser info Redirect gebeurt pas op klik Geen ongewenst gedrag

Mini-opdracht

Maak een “Device Check Panel”:

Check	Output
Is online?	/ ✕
Is touch device?	detect via <code>navigator.maxTouchPoints</code>
Schermtype	Mobiel < 768px, Tablet < 1024px, Desktop anders
Taal	toon taal
OS	zoek "Windows" "Mac" "Linux" in <code>userAgent</code>

UI:

- Knop: “Check device”
- Bootstrap list group

IDs

dc_btn
dc_list

Hint

```
navigator.maxTouchPoints > 0
navigator.userAgent.includes("Windows")
```

Mini-opdracht oplossing

HTML

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-info text-white">Mini-opdracht: Device
  Check</div>
  <div class="card-body">

    <button id="dc_btn" class="btn btn-info w-100 mb-3">Check device</button>
    <ul id="dc_list" class="list-group"></ul>

  </div>
</div>
```

JS

```
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("dc_btn")?.addEventListener("click", () => {

    const ua = navigator.userAgent;
    const width = window.innerWidth;
```

```

const isTouch = navigator.maxTouchPoints > 0;
const os =
  ua.includes("Windows") ? "Windows" :
  ua.includes("Mac") ? "MacOS" :
  ua.includes("Linux") ? "Linux" :
  "Onbekend";

const device =
  width < 768 ? "📱 Mobiel" :
  width < 1024 ? "➡️📱 Tablet" :
  "💻 Desktop";

const info = [
  `Online: ${navigator.onLine ? " Ja" : "❌ Nee"}`,
  `Touch: ${isTouch ? "" : "❌"}`,
  `Device: ${device}`,
  `Taal: ${navigator.language}`,
  `Besturingssysteem: ${os}`
];

document.getElementById("dc_list").innerHTML =
  info.map(i => `<li class="list-group-item">${i}</li>`).join("");
});
});

```

Samenvatting

Concept	Jij kan nu
BOM basics	window, screen, navigator, location
User Agent	OS + browser check
Responsive check	width based detection
Online check	navigator.onLine
Touch detection	mobile/tablet hint
Redirects	bewust & on-click only

Dit is alles wat je nodig hebt om:

- responsive logic te bouwen
- platform checks te doen
- progressive web apps te begrijpen
- device conditional UI te maken

Hoofdstuk 23: Cookies & LocalStorage

Leerdoelen

Na dit hoofdstuk kan je:

- het verschil uitleggen tussen cookies en localStorage
- data opslaan in de browser
- data uitlezen, wijzigen en verwijderen
- JSON gebruiken voor storage
- een mini-app maken die data onthoudt na refresh
- begrijpen waarom security & privacy belangrijk zijn

Dit is de basis voor:

- login remember me
- dark/light mode opslag
- winkelmand-persistentie
- voorkeuren opslaan
- kleine apps zonder backend

Cookies vs LocalStorage

Eigenschap	Cookies	LocalStorage
Capaciteit	Klein (~4kb)	Veel (~5-10MB)
Verloopt	Ja	Blijft tot verwijderd
Stuurt mee met server requests		✗
Gebruik	auth/session	settings/data/apps

In moderne front-end apps gebruiken we **localStorage** tenzij we **auth** moeten opslaan.

LocalStorage Methods

Code	Betekenis
<code>localStorage.setItem(key, value)</code>	opslaan
<code>localStorage.getItem(key)</code>	ophalen
<code>localStorage.removeItem(key)</code>	verwijderen
<code>localStorage.clear()</code>	alles leeg

⚠ Alleen **strings** mogen erin. Daarom gebruiken we JSON.

JSON voor storage

```
localStorage.setItem("data", JSON.stringify(obj));  
const data = JSON.parse(localStorage.getItem("data"));
```

Demo: Naam onthouden

We maken:

- input: naam
- knop: "Opslaan"
- bij laden: toon opgeslagen naam

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">
    Hoofdstuk 23: LocalStorage
  </div>
  <div class="card-body">

    <input id="ls_name" class="form-control mb-2" placeholder="Naam">
    <button id="ls_save" class="btn btn-primary w-100 mb-2">Opslaan</button>

    <div id="ls_output" class="alert alert-secondary mb-0">
      Geen naam opgeslagen
    </div>

  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 23: LocalStorage Demo
// -----

function loadName() {
  const saved = localStorage.getItem("username");
  const out = document.getElementById("ls_output");

  if (saved) {
    out.className = "alert alert-success mb-0";
    out.textContent = `Welkom terug, ${saved}!`;
  }
}

function saveName() {
  const inp = document.getElementById("ls_name").value.trim();
  const out = document.getElementById("ls_output");

  if (!inp) return;

  localStorage.setItem("username", inp);

  out.className = "alert alert-success mb-0";
  out.textContent = ` Naam opgeslagen: ${inp}`;
}

document.addEventListener("DOMContentLoaded", () => {
  loadName();
});
```

```

    document.getElementById("ls_save")
      ?.addEventListener("click", saveName);
  });

```

Naam blijft staan na refresh Volledige UI feedback Beginner-friendly JSON voorbeeld volgt hieronder

Demo 2: Array opslaan (favorietenlijst)

We bouwen:

- input
- add knop
- lijst
- **sla lijst op in localStorage**
- laad terug in bij refresh

index.html

```

<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Favorietenlijst</div>
  <div class="card-body">

    <input id="fav_input" class="form-control mb-2" placeholder="Voeg
favoriet toe">
    <button id="fav_btn" class="btn btn-dark w-100 mb-3">Toevoegen</button>

    <ul id="fav_list" class="list-group mb-2"></ul>

    <button id="fav_clear" class="btn btn-outline-danger w-
100">Wissen</button>

  </div>
</div>

```

main.js

```

let favs = [];

function saveFavs() {
  localStorage.setItem("favs", JSON.stringify(favs));
}

function loadFavs() {
  const saved = JSON.parse(localStorage.getItem("favs"));
  if (saved) favs = saved;
  renderFavs();
}

function renderFavs() {
  document.getElementById("fav_list").innerHTML =
    favs.map(f => `<li class="list-group-item">${f}</li>`).join("");
}

function addFav() {
  const val = document.getElementById("fav_input").value.trim();

```



```

    if (!val) return;

    favs.push(val);
    saveFavs();
    renderFavs();
    document.getElementById("fav_input").value = "";
}

function clearFavs() {
    favs = [];
    localStorage.removeItem("favs");
    renderFavs();
}

document.addEventListener("DOMContentLoaded", () => {
    loadFavs();
    document.getElementById("fav_btn")?.addEventListener("click", addFav);
    document.getElementById("fav_clear")?.addEventListener("click", clearFavs);
});

```

Simple todo system with storage Real-world use (favorites, cart, preferences)

Mini-opdracht

Bouw een **Dark Mode toggle met opslag**:

Event	Actie
Toggle	body krijgt .dark-mode class
Bewaren	localStorage theme = dark/light
Bij reload	thema wordt geladen

IDs

theme_btn
theme_status

Dark-mode class bestaat al van vorige opdrachten? Zo niet, voeg toe:

```

.dark-mode {
    background: #121212;
    color: white;
}

```

Mini-opdracht oplossing

```
function applyTheme(mode) {
  const body = document.body;
  const btn = document.getElementById("theme_btn");
  const status = document.getElementById("theme_status");

  if (mode === "dark") {
    body.classList.add("dark-mode");
    btn.textContent = "Switch to Light";
    status.textContent = "🌙 Donkere modus actief";
  } else {
    body.classList.remove("dark-mode");
    btn.textContent = "Switch to Dark";
    status.textContent = "☀️ Licht modus actief";
  }

  localStorage.setItem("theme", mode);
}

document.addEventListener("DOMContentLoaded", () => {
  applyTheme(localStorage.getItem("theme") || "light");

  document.getElementById("theme_btn")
    ?.addEventListener("click", () => {
      const current = localStorage.getItem("theme");
      applyTheme(current === "dark" ? "light" : "dark");
    });
});
```

Samenvatting

Concept	Wat je nu kan
Cookies	begrijpen waar ze voor dienen
LocalStorage	data lokaal bewaren
JSON storage	objecten & arrays opslaan
Persistent UI	instellingen blijven
Dark Mode	store & load UX

Jij maakt nu **stateful apps**.

Hoofdstuk 24: AJAX & API's Intro

Leerdoelen

Na dit hoofdstuk kan je:

- uitleggen wat AJAX betekent
- `fetch()` gebruiken om data op te halen
- JSON responses verwerken
- verschillen tussen **synchronous** vs **asynchronous** begrijpen
- errors afhandelen
- data tonen in Bootstrap UI
- werken met een echte API endpoint

Na dit hoofdstuk weet je hoe websites “data van internet halen”.

Theorie

Wat is AJAX?

Asynchronous JavaScript And XML (*vroeger, nu meestal JSON*)

Maar gewoon mensentaal:

Data halen van een server zonder pagina te herladen.

Classic vs Modern

Oud	Nieuw
XMLHttpRequest	<code>fetch()</code>
Callbacks	Promises & <code>async/await</code>
XML	JSON

Wij gebruiken **moderne fetch**.

Sync vs Async

Type	Uitleg
Synchronous	code wacht, browser blokkeert
Asynchronous	code loopt door, data komt later

Jouw motto vanaf nu:

Wachten is voor koffiemachines, niet voor JavaScript.

Voorbeeld van async denken

Denk aan pizza bestellen:

- Bestelling plaatsen = **fetch**
- Jij wacht niet bij de oven = **async**
- Pizza komt later = **Promise**
- Jij reageert bij levering = **.then()**
- Pizzakoerier crasht = **catch error**

Demo: Data ophalen met fetch()

We tonen random gebruikers van een **public API**.

API: <https://jsonplaceholder.typicode.com/users>

index.html

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-primary text-white">
    Hoofdstuk 24: Fetch API Demo
  </div>
  <div class="card-body">

    <button id="api_btn" class="btn btn-primary w-100 mb-3">Laad
gebruikers</button>

    <ul id="api_list" class="list-group mb-2"></ul>

    <div id="api_status" class="alert alert-secondary mb-0">
      Klik op de knop om data te laden
    </div>

  </div>
</div>
```

main.js

```
// -----
// Hoofdstuk 24: Fetch API Demo
// -----

async function loadUsers() {
  const status = document.getElementById("api_status");
  const list = document.getElementById("api_list");

  try {
    status.className = "alert alert-warning mb-0";
    status.textContent = "⌚ Data laden...";

    const res = await fetch("https://jsonplaceholder.typicode.com/users");

    if (!res.ok) throw new Error("Netwerk fout");

    const users = await res.json();
```

```

    list.innerHTML = users
      .map(u => `<li class="list-group-
item"><strong>${u.name}</strong><br>${u.email}</li>`)
      .join("");

    status.className = "alert alert-success mb-0";
    status.textContent = ` ${users.length} gebruikers geladen`;

  } catch (err) {
    status.className = "alert alert-danger mb-0";
    status.textContent = "✖ Fout bij laden van data";
  }
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("api_btn")
    ?.addEventListener("click", loadUsers);
});

```

Loading state Error handling Data → UI laatlijst Async/await Public API gebruikt

Wat leerde je eigenlijk?

Skill	Betekenis
fetch()	data van server halen
await	wacht zonder browser te blokkeren
UI states	gebruiker feedback tonen
Promise error handling	professioneel gedrag

Mini-opdracht

Maak een “Random Joke Loader” app:

API: <https://icanhazdadjoke.com/>

Headers verplicht:

```

fetch("https://icanhazdadjoke.com/", {
  headers: { "Accept": "application/json" }
});

```

UI vereisten

Element	ID
knop	joke_btn
joke output	joke_out
status	joke_status

Output format:

😊 hier verschijnt de mop

Laat **loading** en **error** states zien.

Samenvatting

Concept	Jij kan nu
AJAX	data zonder reload
fetch()	JSON data ophalen
async/await	nette async code
error handling	try / catch
DOM rendering	data → UI lijst

Dit is **de kern van modern webdevelopment**. React, Vue, Laravel APIs, Node apps... allemaal hierop gebouwd.

Hoofdstuk 25: JSON File Load + Local API Simulator

Leerdoelen

Na dit hoofdstuk kan je:

- JSON-bestanden lokaal inladen
- JSON parsen en tonen als UI
- een lokale “fake API” simuleren
- laden van data + error states tonen
- basis leggen voor **CRUD** (volgt in volgende hoofdstukken)

Dit voelt als het oefenen van API calls... zonder backend stress.

Wat is JSON?

Eigenschap	Betekenis
JavaScript Object Notation	Data formaat (menselijk leesbaar)
Wordt gebruikt door API's	99% van moderne webservices
Goed te lezen	
Goed te parsen	<code>JSON.parse()</code>

Voorbeeld:

```
[  
  { "name": "Tom", "age": 30 },  
  { "name": "Sara", "age": 26 }  
]
```

JSON in Vite project plaatsen

Maak een bestand:

src/data/users.json

```
[  
  { "name": "Tom", "email": "tom@example.com" },  
  { "name": "Sara", "email": "sara@example.com" },  
  { "name": "Liam", "email": "liam@example.com" }  
]
```

UI: Lokaal JSON bestand laden

index.html

Voeg dit blok toe:

```
<div class="card shadow-sm mt-4">  
  <div class="card-header bg-info text-white">  
    Hoofdstuk 25: Lokale JSON laden  
  </div>  
  <div class="card-body">  
  
    <button id="json_btn" class="btn btn-info w-100 mb-3">  
      Laad lokale gebruikers  
    </button>  
  
    <ul id="json_list" class="list-group mb-2"></ul>  
  
    <div id="json_status" class="alert alert-secondary mb-0">  
      Wacht op actie...  
    </div>  
  
  </div>  
</div>
```

```

main.js
// -----
// Hoofdstuk 25: JSON File Load (Local API Simulator)
// -----

async function loadLocalUsers() {
  const status = document.getElementById("json_status");
  const list = document.getElementById("json_list");

  try {
    status.className = "alert alert-warning mb-0";
    status.textContent = "⌚ JSON laden...";

    const res = await fetch("./data/users.json");
    if (!res.ok) throw new Error("Fout bij laden JSON");

    const users = await res.json();

    list.innerHTML = users
      .map(u => `<li class="list-group-item"><strong>${u.name}</strong><br>${u.email}</li>`)
      .join("");

    status.className = "alert alert-success mb-0";
    status.textContent = ` ${users.length} gebruikers geladen`;

  } catch(err) {
    status.className = "alert alert-danger mb-0";
    status.textContent = "✗ Kon JSON niet laden";
  }
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("json_btn")
    ?.addEventListener("click", loadLocalUsers);
});

```

lokaal JSON laden in Vite loading + success + error UI toont data zoals echte API

UI resultaat

- student klikt *"Laad lokale gebruikers"*
- lijst toont gebruikers
- voelt als database fetch

Mini-opdracht

Maak een **lokale producten database**:

JSON bestand: data/products.json

```
[
  { "name": "Laptop", "price": 899 },
  { "name": "Monitor", "price": 199 },
  { "name": "Toetsenbord", "price": 49 }
]
```

UI features

Element	Beschrijving
Knop	"Laad producten"
List	prijs + naam
Status	loading, success, error

IDs

prod_load_btn
prod_list_local
prod_status_local

Mini-opdracht oplossing (docentversie)

HTML-block

```
<div class="card shadow-sm mt-4">
  <div class="card-header bg-dark text-white">Lokale Producten</div>
  <div class="card-body">

    <button id="prod_load_btn" class="btn btn-dark w-100 mb-3">Laad
    producten</button>

    <ul id="prod_list_local" class="list-group mb-2"></ul>

    <div id="prod_status_local" class="alert alert-secondary mb-0">
      Wacht op actie...
    </div>

  </div>
</div>
```

JS-block

```
async function loadLocalProducts() {
  const list = document.getElementById("prod_list_local");
  const status = document.getElementById("prod_status_local");

  try {
    status.className = "alert alert-warning mb-0";
    status.textContent = "⌛ Laden...";

    const res = await fetch("./data/products.json");
    if (!res.ok) throw new Error("Load error");
```

```

const items = await res.json();

list.innerHTML = items
  .map(p => `<li class="list-group-item">💰 ${p.name} –
€${p.price}</li>`)
  .join("");

status.className = "alert alert-success mb-0";
status.textContent = ` ${items.length} producten geladen`;

} catch {
  status.className = "alert alert-danger mb-0";
  status.textContent = "❌ Fout bij laden";
}
}

document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("prod_load_btn")
    ?.addEventListener("click", loadLocalProducts);
});

```

Samenvatting

Skill	Jij beheerst nu
-------	-----------------

JSON bestandsload	
-------------------	--

fetch lokaal	
--------------	--

renderen naar UI	
------------------	--

error UI	
----------	--

async thinking	
----------------	--

Simulated backend	
-------------------	--

Je hebt nu alle basis om:

- mini backend te simuleren
- API responses te begrijpen
- persistent data systems te bouwen
- later Laravel API / Node API te koppelen